

Министерство науки и высшего образования Российской Федерации  
ФБГОУ ВО «Кубанский государственный технологический университет»  
(ФГБОУ ВО «КубГТУ»)

Факультет информационных технологий и кибербезопасности  
Кафедра информационных систем и программирования  
Направление подготовки 09.03.04 программная инженерия  
Профиль проектирование и разработка программного обеспечения

**КУРСОВОЙ ПРОЕКТ**

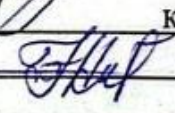
по дисциплине «Проектирование программных комплексов»  
на тему: «Проектирование и разработка информационной системы  
гарантированной доставки сообщений в банковской сфере»

Выполнил студент Милюкин Святослав Николаевич курса 4  
группы 22-КБ-ПР1

Допущен к защите 13.04.2026

Руководитель (нормоконтролёр) работы  Савинская Д.Н.

Защищен 17.04.2026 Оценка отлично

Члены комиссии  канд. техн. наук, доцент Мурлин А.Г.  
 ст. преподаватель Кушнир Н.В.

Краснодар

2026 г.

ФБГОУ ВО «Кубанский государственный технологический университет»  
(ФГБОУ ВО «КубГТУ»)

Факультет информационных технологий и кибербезопасности

Кафедра информационных систем и программирования

Направление подготовки 09.03.04 программная инженерия

Профиль проектирование и разработка программного обеспечения

УТВЕРЖДАЮ

Зав. кафедрой информационных систем и программирования

М.В. Янаева  
«05» 03 2026 г.

**ЗАДАНИЕ**

на курсовой проект

Студенту Милькину Святославу Николаевичу группы 22-КБ-ПР1 4 курса

Тема проекта: «Проектирование и разработка информационной системы  
гарантированной доставки сообщений в банковской сфере»

(утверждена указанием директора института № 82 от «05» 03 2026 г.)

План проекта:

1. Исследование предметной области
2. Спецификация требований.
3. Проектирование приложения.
4. Реализация программы

Объем проекта:

а) пояснительная записка 47 с.

б) иллюстративная часть 20 лист(-ов).

Рекомендуемая литература:

1. Назаров, С. В. Архитектура и проектирование программных систем : монография / С.В. Назаров. - Москва : ИНФРА-М, 2023. — 374 с. ISBN 978-5-16-011753-9. - <https://znanium.com/catalog/product/1895672>

2. Заботина, Н. Н. Проектирование информационных систем : учебное пособие. — Москва : ИНФРА-М, 2022. — 331 с.— DOI 10.12737/2519. - ISBN 978-5-16-004509-2. <https://znanium.com/catalog/product/1840494>

Срок выполнения работы: с «05» 03 2026 по «18» 04 2026 г.

Срок защиты: «14» 04 2026 г.

Дата выдачи задания: «05» 03 2026 г.

Дата сдачи проекта на кафедру: «12» 04 2026 г.

Руководитель работы Савинская Д.Н.

Задание принял студент Милькин С. Н.



## Реферат

Пояснительная записка к курсовой работе содержит: 77 страниц, 20 рисунков, 7 таблиц, 6 источников, 2 приложения.

Курсовая работа выполнена с использованием технологий: KOTLIN 1.9.25, JAVA 21, SPRING BOOT 3.5, POSTGRESQL 16, APACHE KAFKA, REST API, LIQUIBASE, TESTCONTAINERS, DOCKER, UML, PLANTUML.

Объект исследования – событийно-ориентированная банковская платформа для обработки финансовых транзакций между двумя микросервисами.

Цель исследования – спроектировать и реализовать архитектуру микросервисной банковской платформы на основе паттернов CQRS и Transactional Outbox, обеспечивающую надёжную обработку финансовых операций, гарантированную доставку событий через Apache Kafka и согласованность данных между сервисами записи и чтения.

Предмет исследования – проектирование архитектуры, модели данных, механизмов надёжной доставки событий и обеспечения идемпотентности в распределённой банковской системе.

Практическая значимость состоит в реализации готовой к эксплуатации платформы с полным покрытием автоматическими тестами, документированным API и справочными материалами для развёртывания и сопровождения.

## Содержание

|                                                                                                                    |    |
|--------------------------------------------------------------------------------------------------------------------|----|
| Реферат .....                                                                                                      | 1  |
| Введение.....                                                                                                      | 7  |
| 1 Нормативные ссылки.....                                                                                          | 9  |
| 2 Описание предметной области .....                                                                                | 10 |
| 2.1 Общее описание предметной области .....                                                                        | 10 |
| 2.2 Назначение, цели и задачи создания ПО .....                                                                    | 10 |
| 2.3 Требования к структуре ПО и базе данных.....                                                                   | 11 |
| 2.4 Описание входных документов и сообщений.....                                                                   | 11 |
| 2.5 Описание выходных документов и сообщений .....                                                                 | 12 |
| 2.6 Список ограничений .....                                                                                       | 12 |
| 3 Разработка технического задания ПО.....                                                                          | 14 |
| 3.1 Общие сведения .....                                                                                           | 14 |
| 3.2 Назначение и цели создания (развития) системы.....                                                             | 14 |
| 3.3 Характеристика объектов автоматизации .....                                                                    | 14 |
| 3.4 Требования к системе .....                                                                                     | 15 |
| 3.4.1 Требования к внешним интерфейсам .....                                                                       | 15 |
| 3.4.2 Функциональные требования .....                                                                              | 16 |
| 3.4.3 Нефункциональные требования .....                                                                            | 17 |
| 3.5 Состав и содержание работ по созданию системы.....                                                             | 18 |
| 3.6 Порядок контроля и приемки системы.....                                                                        | 18 |
| 3.7 Требования к составу и содержанию работ по подготовке объекта<br>автоматизации к вводу системы в действие..... | 19 |
| 3.8 Требования к документированию .....                                                                            | 19 |
| 3.9 Источники разработки.....                                                                                      | 20 |
| 4 Проектирование структуры базы данных.....                                                                        | 21 |
| 4.1 Описание сущностей .....                                                                                       | 21 |
| 4.2 Построение концептуальной схемы базы данных.....                                                               | 23 |
| 5 Реализация базы данных средствами выбранной СУБД .....                                                           | 24 |
| 5.1 Выбор СУБД.....                                                                                                | 24 |
| 5.2 Проектирование таблиц.....                                                                                     | 24 |
| 5.3 Построение ER-диаграммы.....                                                                                   | 26 |

|      |                                                               |    |
|------|---------------------------------------------------------------|----|
| 6    | Проектирование структуры приложения.....                      | 28 |
| 6.1  | Построение структурной схемы ПО .....                         | 28 |
| 6.2  | Построение архитектуры информационной системы .....           | 29 |
| 6.3  | Разработка функциональной модели .....                        | 29 |
| 6.4  | Диаграмма компонентов .....                                   | 31 |
| 6.5  | Диаграммы кооперации.....                                     | 32 |
| 6.6  | Диаграмма размещения .....                                    | 33 |
| 6.7  | Диаграмма состояний .....                                     | 34 |
| 6.8  | Диаграммы потоков данных .....                                | 34 |
| 6.9  | Диаграмма классов.....                                        | 35 |
| 6.10 | Диаграмма последовательности .....                            | 37 |
| 6.11 | Диаграмма прецедентов .....                                   | 37 |
| 7    | Описание алгоритмов обработки данных.....                     | 39 |
| 7.1  | Описание алгоритмов обработки данных .....                    | 39 |
| 7.2  | Блок-схемы алгоритмов обработки данных .....                  | 40 |
| 8    | Описание реализации программного продукта .....               | 44 |
| 8.1  | Выбор среды программирования .....                            | 44 |
| 8.2  | Последовательность разработки прикладной программы.....       | 44 |
| 8.3  | Схема меню приложения .....                                   | 45 |
| 8.4  | Описание окон приложения.....                                 | 46 |
| 8.5  | Функциональные возможности программного приложения.....       | 47 |
| 8.6  | Функции и процедуры обработки данных .....                    | 48 |
| 9    | Тестирование программного продукта и отладка.....             | 50 |
| 9.1  | Стратегия тестирования .....                                  | 50 |
| 9.2  | Инструменты тестирования .....                                | 50 |
| 9.3  | Инструменты тестирования .....                                | 51 |
| 9.4  | Тестовые утилиты .....                                        | 52 |
| 9.5  | Модульные тесты.....                                          | 52 |
| 9.6  | Интеграционные тесты.....                                     | 53 |
| 9.7  | Запуск и результаты тестирования.....                         | 55 |
| 10   | Проектирование технической и справочной информации к ПО ..... | 57 |
| 10.1 | Требования к окружению и подключение.....                     | 57 |

|                                                     |    |
|-----------------------------------------------------|----|
| 10.2 Основные REST endpoint.....                    | 58 |
| 10.3 Формат сообщений для Kafka.....                | 60 |
| 10.4 Реализация паттерна Transactional Outbox ..... | 61 |
| 10.5 Мониторинг .....                               | 62 |
| Вывод.....                                          | 64 |
| Список используемой литературы .....                | 66 |
| Приложение 1 Листинг программы .....                | 67 |
| Приложение 2 Проверка на антиплагиат .....          | 77 |

## Введение

Выбор темы выпускной квалификационной работы связан с тем, что современные банковские и финансовые системы всё чаще переходят от монолитных решений к распределённым архитектурам. Обработка платежей, переводов и истории операций — это именно та область, где ошибки обходятся дорого: потеря транзакции или двойное списание недопустимы. Мне хотелось разобраться, как крупные компании решают эти задачи на практике, а не в теории.

До начала этой работы я уже имел опыт с Spring Boot и PostgreSQL, но никогда не строил систему, в которой два сервиса общаются асинхронно через брокера сообщений. Apache Kafka сейчас является одним из наиболее востребованных инструментов на рынке: с её помощью строят надёжные потоки событий между компонентами, обеспечивают масштабируемость и отказоустойчивость. Поэтому я решил использовать именно её — на понятном и прикладном сценарии банковских транзакций.

В процессе проектирования выяснилось, что переход к асинхронному взаимодействию порождает нетривиальную проблему: запись в базу данных и публикация события в Kafka не могут быть выполнены атомарно. Если сервис упадёт между этими двумя операциями, событие будет потеряно, хотя транзакция уже зафиксирована — или наоборот. Именно для решения этой проблемы я применил паттерн Transactional Outbox: событие записывается в ту же базу данных в рамках одной транзакции с бизнес-данными, а отдельный процесс доставляет его в Kafka. Это гарантирует, что ни одно событие не будет потеряно даже при сбое.

Второй ключевой идеей стал паттерн CQRS — разделение операций записи и чтения на два независимых сервиса. Сервис ``core-transaction`` принимает команды: создаёт переводы, обрабатывает платежи, управляет счетами. Сервис ``core-ledger`` отвечает только за чтение: потребляет события из Kafka и предоставляет историю транзакций с фильтрацией и выгрузкой

отчётов в форматах PDF, DOCX и HTML. Такое разделение позволяет независимо масштабировать нагрузку на запись и чтение, а также изолировать сбои.

Цель выпускной квалификационной работы — спроектировать и реализовать событийно-управляемую платформу обработки банковских транзакций на основе архитектуры CQRS с применением паттерна Transactional Outbox.

Для достижения цели были поставлены следующие задачи:

1. описать предметную область и сформировать техническое задание на разработку системы;
2. спроектировать базы данных обоих сервисов и реализовать схему миграций через Liquibase;
3. реализовать сервис core-transaction с поддержкой различных типов банковских операций и механизмом Transactional Outbox;
4. реализовать сервис core-ledger с идемпотентным потреблением Kafka-событий и генерацией отчётов;
5. разработать UML-диаграммы, отражающие архитектуру и поведение системы;
6. описать ключевые алгоритмы обработки данных и покрыть реализацию автоматизированными тестами.



## **1 Нормативные ссылки**

В данных методических указаниях использованы ссылки на следующие нормативные документы:

- ГОСТ Р7.0.5-2008 СИБИД. Библиографическая ссылка. Общие требования и правила составления;
- ГОСТ Р1.5-2004. Стандарты национальные РФ. Правила построения, изложения, оформления и обозначения;
- ГОСТ 2.301-68 ЕСКД. Форматы;
- ГОСТ 7.82-2001 СИБИД. Библиографическая запись. Библиографическое описание электронных ресурсов. Общие требования и правила составления;
- ГОСТ 7.9-95 СИБИД. Реферат и аннотация. Общие требования.

## **2 Описание предметной области**

### **2.1 Общее описание предметной области**

Предметная область проекта — обработка банковских финансовых операций. Пользователь системы — клиент банка, у которого есть один или несколько счетов: расчётный, сберегательный, депозитный или брокерский. К счёту может быть привязана платёжная карта, используемая при межбанковских переводах. Основное действие в системе — транзакция: событие, которое изменяет баланс одного или двух счетов.

Система поддерживает несколько типов операций: внутренние переводы между счетами, переводы по номеру телефона через СБП, межбанковские переводы по номеру карты, зачисление средств без списания (денежный подарок и компенсация), а также погашение кредита. Помимо выполнения операций, система ведёт полную историю транзакций и позволяет выгружать её в виде отчёта.

Ключевая сложность предметной области состоит в том, что банковская операция должна быть зафиксирована ровно один раз — без потерь и без дублирования. Система устроена так, что запись транзакции в базу данных и уведомление об этом смежного сервиса происходят атомарно: либо оба действия выполняются вместе, либо не выполняется ни одно из них.

### **2.2 Назначение, цели и задачи создания ПО**

Назначение системы — надёжная обработка банковских транзакций с разделением операций записи и чтения на два независимых сервиса.

Цель — разработать и оформить документацию к платформе, которая гарантирует согласованность данных при асинхронном взаимодействии сервисов и корректно обрабатывает повторные или дублирующие запросы.

Для достижения цели поставлены следующие задачи:

- определить требования к системе и её интерфейсам;
- спроектировать базы данных обоих сервисов;

- разработать архитектуру приложения и подготовить UML-диаграммы;
- описать алгоритмы обработки транзакций и доставки событий между сервисами;
- зафиксировать поведение системы при недоступности брокера сообщений и при повторной доставке событий.

## **2.3 Требования к структуре ПО и базе данных**

Система состоит из двух микросервисов с чёткими зонами ответственности, каждый из которых имеет собственную базу данных.

Первый сервис отвечает за приём и выполнение команд: создание пользователей и счетов, проведение финансовых операций. После успешного выполнения операции сервис формирует событие и гарантированно доставляет его второму сервису через брокер сообщений. Если брокер в этот момент недоступен, событие не теряется — оно сохраняется в базе данных и будет отправлено при первой возможности.

Второй сервис отвечает исключительно за чтение: он получает события от первого сервиса, сохраняет их в собственную базу данных и предоставляет REST API для запроса истории транзакций и формирования отчётов. Прямая запись через API в этот сервис не предусмотрена.

## **2.4 Описание входных документов и сообщений**

Входными воздействиями для первого сервиса являются HTTP-запросы к REST API. Пользователь отправляет команды: зарегистрировать клиента, открыть счёт, выполнить финансовую операцию. Каждый запрос на проведение транзакции содержит уникальный идентификатор операции, сумму, необходимые реквизиты (номера счетов или карт, номер телефона — в зависимости от типа операции) и необязательное описание.

Входными сообщениями для второго сервиса являются события из топика Kafka. Каждое сообщение содержит идентификатор события, тип операции, реквизиты счетов, сумму, валюту, статус транзакции и служебные

метаданные: идентификатор агрегата, ключ партиционирования и время создания события.

## **2.5 Описание выходных документов и сообщений**

Ответами первого сервиса на REST-запросы являются JSON-объекты. При создании пользователя возвращаются его данные и дата регистрации. При открытии счёта — тип счёта, валюта, баланс и сгенерированный номер счёта. При проведении транзакции — тип операции, статус, реквизиты и временные метки.

Ответами второго сервиса являются записи реестра транзакций: ссылка на исходную транзакцию, тип события и операции, реквизиты счетов, сумма, статус, время создания и время получения. Дополнительно сервис формирует отчёты по истории транзакций за выбранный период в форматах PDF, DOCX или HTML.

В случае ошибки оба сервиса возвращают единый формат ответа: HTTP-статус, текст ошибки и временная метка.

## **2.6 Список ограничений**

Основные ограничения системы продиктованы требованиями финансовой консистентности.

Баланс счёта не может стать отрицательным. При недостатке средств операция отклоняется до внесения каких-либо изменений.

Каждый запрос на проведение транзакции должен содержать уникальный идентификатор. Повторный запрос с тем же идентификатором отклоняется, а не выполняется дважды. Аналогичная защита реализована при получении событий из Kafka: повторно доставленное сообщение распознаётся и пропускается.

При переводе между двумя счетами порядок их блокировки всегда фиксирован. Это исключает взаимную блокировку при одновременных встречных переводах между одними и теми же счетами.

Второй сервис является строго read-only с точки зрения внешнего API. Все данные в нём появляются исключительно через обработку событий из Kafka.

## **3 Разработка технического задания ПО**

### **3.1 Общие сведения**

Техническое задание описывает функциональные и нефункциональные требования к событийно-управляемой платформе обработки банковских транзакций, реализованной в виде двух микросервисов: core-transaction и core-ledger. Система предоставляет REST API для выполнения финансовых операций, а также для запроса истории транзакций и формирования отчётов. Взаимодействие между сервисами осуществляется асинхронно через брокер сообщений Apache Kafka.

Документ предназначен для разработчиков, аналитиков и специалистов по тестированию, принимающих участие в создании и сопровождении системы.

### **3.2 Назначение и цели создания (развития) системы**

Система предназначена для надёжной обработки банковских транзакций с разделением операций записи и чтения, гарантированной доставки событий между сервисами и идемпотентного ведения реестра транзакций.

Цель данной работы — собрать и оформить проектную документацию по заданной структуре. В рамках этой цели описана предметная область, сформировано техническое задание, спроектированы базы данных и UML-диаграммы, а также зафиксированы ключевые алгоритмы обработки транзакций и поведение системы при частичных сбоях инфраструктуры.

### **3.3 Характеристика объектов автоматизации**

Объектами автоматизации в системе являются следующие сущности предметной области:

- клиент банка и его персональные данные;
- банковский счёт с типом, валютой и балансом;



- платёжная карта, привязанная к счёту;
- финансовая транзакция и её атрибуты: тип, статус, сумма, реквизиты сторон;
- очередь гарантированной доставки событий (Outbox) и факты публикации в брокер;
- запись реестра транзакций — денормализованная read-модель на стороне core-ledger.

Дополнительным объектом автоматизации является сценарий асинхронной доставки: отправка события считается завершённой только после подтверждения успешной записи в базу данных core-ledger, а не в момент публикации в Kafka.

### **3.4 Требования к системе**

#### **3.4.1 Требования к внешним интерфейсам**

1. Интерфейс пользователя. Взаимодействие с системой осуществляется через REST API по протоколу HTTP. Для каждого сервиса доступен Swagger UI (/swagger-ui.html), предоставляющий интерактивную документацию по всем эндпоинтам с описанием полей запросов и ответов.

2. Интерфейсы технической инфраструктуры. Каждый сервис работает с собственной базой данных PostgreSQL 16. Kafka-кластер из трёх брокеров обеспечивает асинхронное взаимодействие между сервисами. Все компоненты запускаются в Docker-контейнерах. Метрики обоих сервисов экспонируются в формате Prometheus через эндпоинт /actuator/prometheus.

3. Программные интерфейсы. core-transaction предоставляет REST API для операций записи: управление пользователями и счетами, проведение транзакций. core-ledger предоставляет REST API только для операций чтения: запрос истории транзакций с фильтрацией и выгрузка отчётов. Взаимодействие между сервисами — через Kafka-топик core.transactions.request.

4. Формат взаимодействия. Все запросы и ответы REST API передаются в формате JSON. Kafka-сообщения сериализуются в JSON. При ошибках оба сервиса возвращают единый структурированный ответ с HTTP-статусом, текстом ошибки и временной меткой.

### 3.4.2 Функциональные требования

Таблица 1 – Функциональные требования

| №     | Требование                                                                                                                                                           |
|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ФТ-1  | Система должна обеспечивать создание клиента банка с указанием имени, фамилии, номера телефона и электронной почты.                                                  |
| ФТ-2  | Система должна обеспечивать открытие счёта выбранного типа (расчётный, сберегательный, депозитный, брокерский) с автоматической генерацией уникального номера счёта. |
| ФТ-3  | Система должна обеспечивать выполнение внутреннего перевода между двумя счетами с одновременным списанием у отправителя и зачислением получателю.                    |
| ФТ-4  | Система должна обеспечивать выполнение перевода по номеру телефона через СБП.                                                                                        |
| ФТ-5  | Система должна обеспечивать выполнение межбанковского перевода по номерам платёжных карт.                                                                            |
| ФТ-6  | Система должна обеспечивать зачисление средств на счёт без списания у отправителя (денежный подарок, компенсация).                                                   |
| ФТ-7  | Система должна обеспечивать погашение кредита: списание с расчётного счёта клиента и зачисление на счёт назначения.                                                  |
| ФТ-8  | После выполнения любой финансовой операции система должна атомарно сохранять событие в очередь доставки и гарантировать его публикацию в брокер сообщений.           |
| ФТ-9  | Система должна предоставлять историю транзакций по счёту с фильтрацией по дате, типу операции и статусу, а также с поддержкой постраничной навигации.                |
| ФТ-10 | Система должна формировать отчёт по истории транзакций за выбранный период в одном из форматов: PDF, DOCX или HTML.                                                  |

Представленные функциональные требования охватывают полный жизненный цикл банковской операции: от регистрации клиента и открытия счёта до проведения транзакции, гарантированной доставки события в реестр и формирования итоговой отчётности.

### 3.4.3 Нефункциональные требования

Таблица 2 – Нефункциональные требования

| №     | Требование                                                                                                                                                                                       |
|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| НФТ-1 | Баланс счёта не может стать отрицательным. Операция, приводящая к отрицательному балансу, должна быть отклонена до внесения каких-либо изменений в базу данных.                                  |
| НФТ-2 | Каждый запрос на проведение транзакции должен содержать уникальный идентификатор. Повторный запрос с тем же идентификатором должен быть отклонён, а не выполнен повторно.                        |
| НФТ-3 | Повторно доставленное Kafka-сообщение не должно создавать дублирующую запись в реестре. Сервис core-ledger должен распознавать дубликат и пропускать его.                                        |
| НФТ-4 | При одновременном проведении операций по одним и тем же счетам система должна исключать взаимную блокировку за счёт фиксированного порядка захвата блокировок.                                   |
| НФТ-5 | Kafka-сообщение должно подтверждаться только после успешной записи в базу данных core-ledger. При сбое запись должна быть повторена в соответствии с настроенной политикой повторных попыток.    |
| НФТ-6 | Все изменения схемы баз данных должны выполняться через версионированные миграции. Запуск сервиса с несоответствующей схемой должен завершаться с ошибкой.                                       |
| НФТ-7 | Каждый сервис должен экспонировать метрики производительности и бизнес-показатели (количество транзакций по типу и статусу, количество дублирующих событий) в формате, совместимом с Prometheus. |

Представленные нефункциональные требования обеспечивают финансовую корректность операций, согласованность данных при асинхронном взаимодействии и наблюдаемость системы в производственной среде.

### **3.5 Состав и содержание работ по созданию системы**

Состав работ включает:

- анализ предметной области и формирование требований к системе;
- проектирование баз данных обоих сервисов и построение ER-диаграмм;
- реализацию сервиса core-transaction: REST API, бизнес-логика операций, компонент гарантированной доставки событий;
- реализацию сервиса core-ledger: Kafka-потребитель с идемпотентной обработкой, REST API истории транзакций, генерация отчётов;
- разработку UML-диаграмм, описывающих архитектуру и поведение системы;
- написание автоматизированных тестов и отладку по результатам;
- оформление пояснительной записки и технической документации.

Перечисленные работы охватывают полный цикл разработки — от анализа требований до тестирования и документирования — и обеспечивают системный подход к созданию платформы.

### **3.6 Порядок контроля и приемки системы**

Приёмка выполняется по итогам проверки ключевых сценариев:

- проведение каждого из поддерживаемых типов транзакций и проверка корректности изменения балансов;
- повторная отправка запроса с тем же идентификатором и проверка, что транзакция не была выполнена дважды;

- проверка появления записи в реестре core-ledger после успешного проведения операции в core-transaction;
- повторная доставка Kafka-сообщения и проверка, что дублирующая запись в реестре не создаётся;
- запрос истории транзакций с фильтрацией и формирование отчёта в каждом из поддерживаемых форматов.

### **3.7 Требования к составу и содержанию работ по подготовке объекта автоматизации к вводу системы в действие**

Подготовка к вводу в действие предполагает развёртывание следующих компонентов инфраструктуры:

- Kafka-кластер из трёх брокеров с созданием топика core.transactions.request;
- база данных transaction\_db для сервиса core-transaction с применением всех миграций;
- база данных ledger\_db для сервиса core-ledger с применением всех миграций.

Все компоненты запускаются через Docker Compose. Сервисы не имеют общих баз данных и не обращаются напрямую к базам данных друг друга.

### **3.8 Требования к документированию**

Документация включает:

- техническое задание с функциональными и нефункциональными требованиями;
- описание сущностей и схем баз данных обоих сервисов;
- архитектурные и поведенческие UML-диаграммы;
- описание алгоритмов обработки транзакций и доставки событий;
- техническую справку: перечень REST-эндпоинтов, формат Kafka-сообщений, инструкция по запуску.

### 3.9 Источники разработки

Источниками разработки послужили:

- учебные материалы по проектированию распределённых систем и паттернам CQRS и Transactional Outbox;
- официальная документация Apache Kafka, Spring Boot и Spring Data JPA;
- стандарт ГОСТ 7.0.5-2008 «Библиографическая ссылка. Общие требования и правила составления»;
- стандарт ГОСТ 1.5-2004 «Стандарты национальные. Правила построения, изложения, оформления и обозначения»;
- стандарт IEEE 830-1998 в части рекомендуемой практики по спецификации программных требований.

В ходе работы составлено техническое задание на систему обработки банковских транзакций с применением архитектуры CQRS и асинхронного взаимодействия через брокер сообщений в соответствии с образцом IEEE 830-1998.

## 4 Проектирование структуры базы данных

### 4.1 Описание сущностей

Проектирование структуры базы данных выполнено на основании требований предметной области. Система использует две независимые базы данных: `transaction_db` обслуживает сервис обработки транзакций, `ledger_db` — сервис ведения реестра. Основные сущности соответствуют таблицам проекта.

Таблица 3 – Назначение сущностей

| Сущность                | Описание                                                                                                                                                                                                                                                                                                         |
|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| users                   | Клиенты банка: идентификатор, имя, фамилия, отчество, номер телефона (уникальный), электронная почта, дата создания и обновления.                                                                                                                                                                                |
| accounts                | Банковские счета: идентификатор, ссылка на клиента, номер счёта (уникальный, 20 символов), тип счёта, валюта, баланс (NUMERIC(19,4), не менее 0), признак активности, даты создания и обновления. Связан с users по user_id.                                                                                     |
| cards                   | Платёжные карты: идентификатор, ссылка на счёт, номер карты (уникальный), дата истечения, признак активности, дата создания. Связана с accounts по account_id.                                                                                                                                                   |
| transactions            | Финансовые транзакции: идентификатор, ключ идемпотентности (UUID, уникальный), тип операции, статус, ссылки на счёт-источник (nullable) и счёт-получатель, сумма, валюта, описание, текст ошибки, дата создания и завершения.                                                                                    |
| outbox_messages         | Очередь событий для гарантированной доставки: идентификатор, тип и идентификатор агрегата, тип события, ключ партиционирования, номер партиции, полезная нагрузка в формате JSONB, дата создания. Составной первичный ключ (id, partition_num). Таблица секционирована по списку: 8 физических партиций (p0–p7). |
| outbox_consumer_offsets | Смещения потребителей Outbox: идентификатор, наименование группы потребителей, номер партиции, последнее обработанное                                                                                                                                                                                            |

| Сущность               | Описание                                                                                                                                                                                                                  |
|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                        | смещение, дата обновления. Уникальное ограничение на пару (consumer_group, partition_num).                                                                                                                                |
| outbox_partition_locks | Блокировки партиций: идентификатор, группа потребителей, номер партиции, идентификатор захватившего блокировку процесса, время захвата и время истечения. Уникальное ограничение на пару (consumer_group, partition_num). |

Таблица 4 — Описание сущностей ledger\_db

| Сущность           | Описание                                                                                                                                                                                                                                                                                                                                                                  |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| transaction_ledger | Денормализованная read-модель реестра транзакций: идентификатор, ключ идемпотентности (уникальный), тип события, идентификатор исходной транзакции, тип операции, номера счетов отправителя (nullable) и получателя, сумма, валюта, статус, идентификатор агрегата, ключ партиционирования, идентификатор Outbox-сообщения, время создания события и время его получения. |

Таблица 5 — Описание связей transaction\_db

| Связь                                            | Описание                                                                                                          |
|--------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| accounts → users (user_id)                       | Счёт принадлежит клиенту; у одного клиента может быть несколько счетов.                                           |
| cards → accounts (account_id)                    | Карта привязана к счёту; у одного счёта может быть несколько карт.                                                |
| transactions → accounts (source_account_id)      | Транзакция ссылается на счёт-источник; поле nullable — для операций без списания (денежный подарок, компенсация). |
| transactions → accounts (destination_account_id) | Транзакция ссылается на счёт-получатель; обязательное поле для всех типов операций.                               |

Сущности outbox\_messages, outbox\_consumer\_offsets и outbox\_partition\_locks не имеют внешних ключей на бизнес-таблицы — это намеренное архитектурное решение: Outbox-компонент работает независимо и не создаёт связей, которые могли бы заблокировать удаление бизнес-данных.



База данных ledger\_db содержит единственную таблицу transaction\_ledger без внешних ключей. Это денормализованная модель: все необходимые данные хранятся в одной строке, что исключает JOIN-запросы при чтении истории и ускоряет выборку.

## 4.2 Построение концептуальной схемы базы данных

Концептуальная схема построена для отображения связей между сущностями предметной области. Поскольку система использует две независимые базы данных, схема разделена на два блока. Связь между ними логическая: transaction\_ledger.source\_transaction\_id ссылается на идентификатор транзакции из transaction\_db, однако на уровне базы данных это ограничение не закреплено.

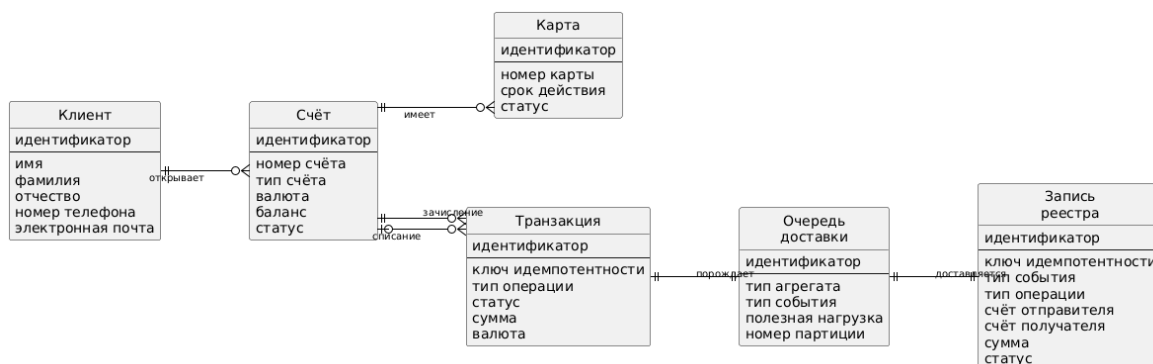


Рисунок 1 – Концептуальная схема

На концептуальной схеме отражены все сущности предметной области. В базе transaction\_db центральной сущностью является transactions, связанная со счетами через два внешних ключа. Сущности outbox\_messages, outbox\_consumer\_offsets и outbox\_partition\_locks обеспечивают работу механизма гарантированной доставки и намеренно не связаны с бизнес-таблицами внешними ключами. База ledger\_db содержит единственную денормализованную таблицу transaction\_ledger, связанную с transaction\_db логически через идентификатор исходной транзакции, передаваемый в Kafka-сообщении.

## 5 Реализация базы данных средствами выбранной СУБД

### 5.1 Выбор СУБД

Для хранения данных обоих сервисов выбрана СУБД PostgreSQL 16. Этот выбор обусловлен несколькими причинами. Во-первых, PostgreSQL обеспечивает надёжную транзакционную модель с полной поддержкой ACID, что критично для банковской предметной области: списание и зачисление должны фиксироваться атомарно. Во-вторых, PostgreSQL поддерживает тип данных JSONB, который используется для хранения полезной нагрузки Outbox-событий — это позволяет сохранять произвольную структуру события без дополнительных таблиц. В-третьих, PostgreSQL поддерживает LIST-секционирование таблиц, что применяется для разделения таблицы `outbox_messages` на 8 физических партиций для параллельной обработки. Наконец, PostgreSQL поддерживает пессимистичные блокировки (`SELECT ... FOR UPDATE`), которые используются для безопасного изменения балансов при конкурентных переводах.

Управление схемой осуществляется через инструмент миграций Liquibase: каждое изменение оформлено как SQL-файл, подключённый через XML-чейнджлог. Hibernate работает в режиме `validate` — приложение не создаёт и не изменяет таблицы самостоятельно, а лишь проверяет соответствие схемы при запуске.

### 5.2 Проектирование таблиц

Проектирование таблиц выполнено с учётом связей между сущностями предметной области. Система использует две изолированные базы данных.

База данных `transaction_db` (порт 5434) содержит семь таблиц и пять последовательностей:

- Таблица `users` хранит данные клиентов банка: имя, фамилия, отчество, номер телефона (уникальный) и электронная почта. Первичный ключ — автоинкрементный `BIGSERIAL`.

- Таблица `accounts` хранит банковские счета со связью с `users` по `user_id`. Номер счёта (`account_number`, 20 символов) генерируется автоматически через PostgreSQL-последовательности: `seq_account_checking`, `seq_account_savings`, `seq_account_deposit`, `seq_account_brokerage` — по одной на каждый тип. Баланс хранится как `NUMERIC(19,4)` с ограничением `CHECK (balance >= 0)`. Допустимые типы счетов ограничены через `CHECK (account_type IN ('CHECKING', 'SAVINGS', 'DEPOSIT', 'BROKERAGE'))`.

- Таблица `cards` хранит платёжные карты со связью с `accounts` по `account_id`. Номер карты уникален, срок действия хранится как `DATE`. Номера карт генерируются через последовательность `seq_card`.

- Таблица `transactions` хранит финансовые операции. Ключ идемпотентности (`idempotency_key`, `UUID`) имеет ограничение `UNIQUE`. Статус ограничен через `CHECK (status IN ('PENDING', 'COMPLETED', 'FAILED', 'CANCELLED'))`. Тип операции ограничен восемью допустимыми значениями. Поле `source_account_id` допускает `NULL` — для операций без списания. Сумма проверяется ограничением `CHECK (amount > 0)`.

- Таблица `outbox_messages` реализует `Transactional Outbox`. Составной первичный ключ (`id`, `partition_num`). Таблица секционирована по списку (`PARTITION BY LIST (partition_num)`) на 8 физических партиций (`outbox_messages_p0` — `outbox_messages_p7`). Полезная нагрузка хранится в поле `payload` типа `JSONB`.

- Таблица `outbox_consumer_offsets` хранит последнее обработанное смещение для каждой пары (`consumer_group`, `partition_num`) с ограничением уникальности.

- Таблица `outbox_partition_locks` реализует лизинговую блокировку партиций: каждая запись содержит идентификатор захватившего процесса, время захвата и время истечения. Уникальное ограничение на пару (`consumer_group`, `partition_num`).

База данных `ledger_db` (порт 5435) содержит одну таблицу:

- Таблица `transaction_ledger` — денормализованная read-модель. Ключ идемпотентности (`idempotency_key`, `VARCHAR(255)`) имеет ограничение `UNIQUE` для защиты от дубликатов. Таблица содержит все необходимые данные в одной строке: тип события, идентификатор исходной транзакции, тип операции, номера счетов, сумму, валюту, статус, метаданные агрегата и временные метки. Проиндексированы поля `source_transaction_id`, `source_account_number`, `destination_account_number` и `event_created_at` для быстрой выборки истории.

### 5.3 Построение ER-диаграммы

ER-диаграмма построена на основании реальных таблиц и их внешних ключей. Поскольку система использует две независимые базы данных, диаграммы представлены отдельно для каждой.

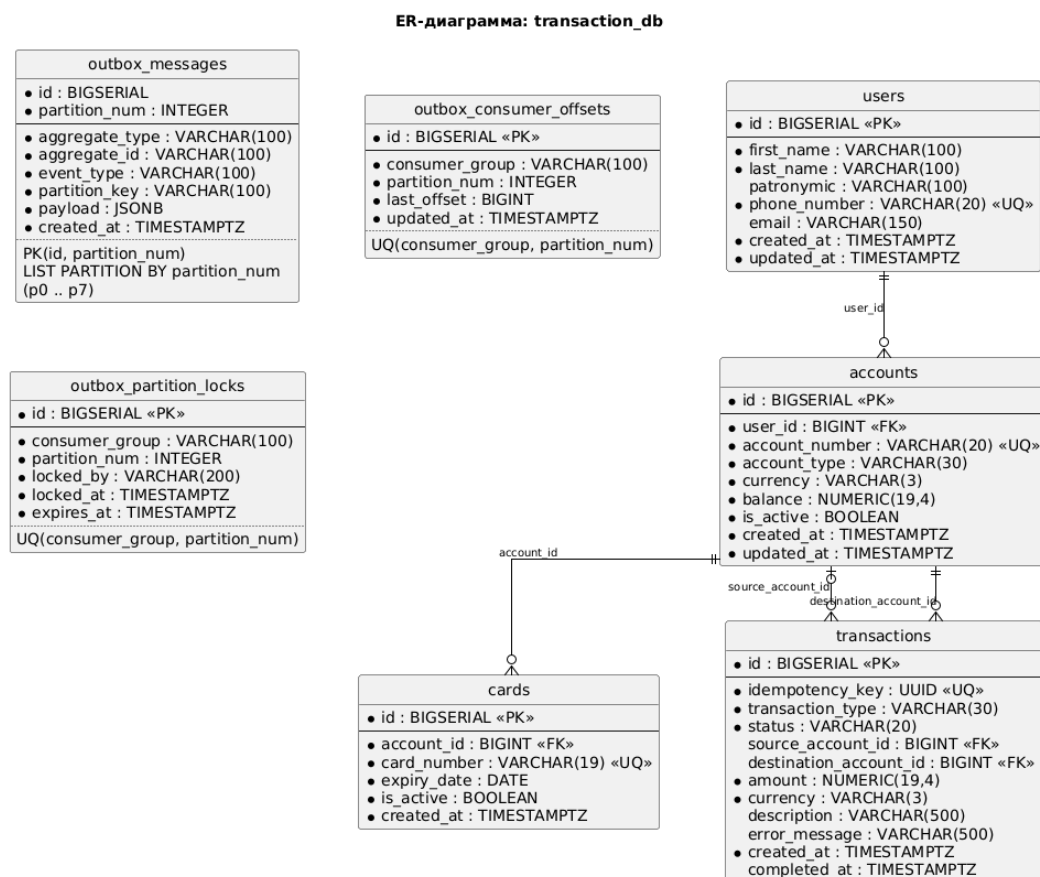


Рисунок 2 – ER-диаграмма `transaction_db`

ER-диаграмма: ledger\_db

| transaction_ledger                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>• id : BIGSERIAL «PK»</li> <li>• idempotency_key : VARCHAR(255) «UQ»</li> <li>• event_type : VARCHAR(50)</li> <li>• source_transaction_id : BIGINT «IDX»</li> <li>• transaction_type : VARCHAR(30)</li> <li>• source_account_number : VARCHAR(20) «IDX»</li> <li>• destination_account_number : VARCHAR(20) «IDX»</li> <li>• amount : NUMERIC(19,4)</li> <li>• currency : VARCHAR(3)</li> <li>• status : VARCHAR(20)</li> <li>• aggregate_id : VARCHAR(255)</li> <li>• partition_key : VARCHAR(255)</li> <li>• source_message_id : BIGINT</li> <li>• event_created_at : TIMESTAMPTZ «IDX»</li> <li>• received_at : TIMESTAMPTZ</li> </ul> |

Рисунок 3 – ER-диаграмма ledger\_db

На ER-диаграмме transaction\_db отображены все семь таблиц и связи между ними. Центральная бизнес-сущность — transactions — связана со счетами двумя внешними ключами: source\_account\_id (nullable для операций без списания) и destination\_account\_id. Счета связаны с клиентами, карты — со счетами. Таблицы Outbox (outbox\_messages, outbox\_consumer\_offsets, outbox\_partition\_locks) намеренно не имеют внешних ключей на бизнес-таблицы: Outbox-компонент работает автономно, и отсутствие FK позволяет очищать старые сообщения без каскадных зависимостей.

## 6 Проектирование структуры приложения

### 6.1 Построение структурной схемы ПО

Диаграмма пакетов в UML показывает крупные блоки системы (пакеты, модули) и зависимости между ними. Она используется, чтобы увидеть декомпозицию проекта на уровне слоёв и подсистем, направление зависимостей и степень связанности между частями.

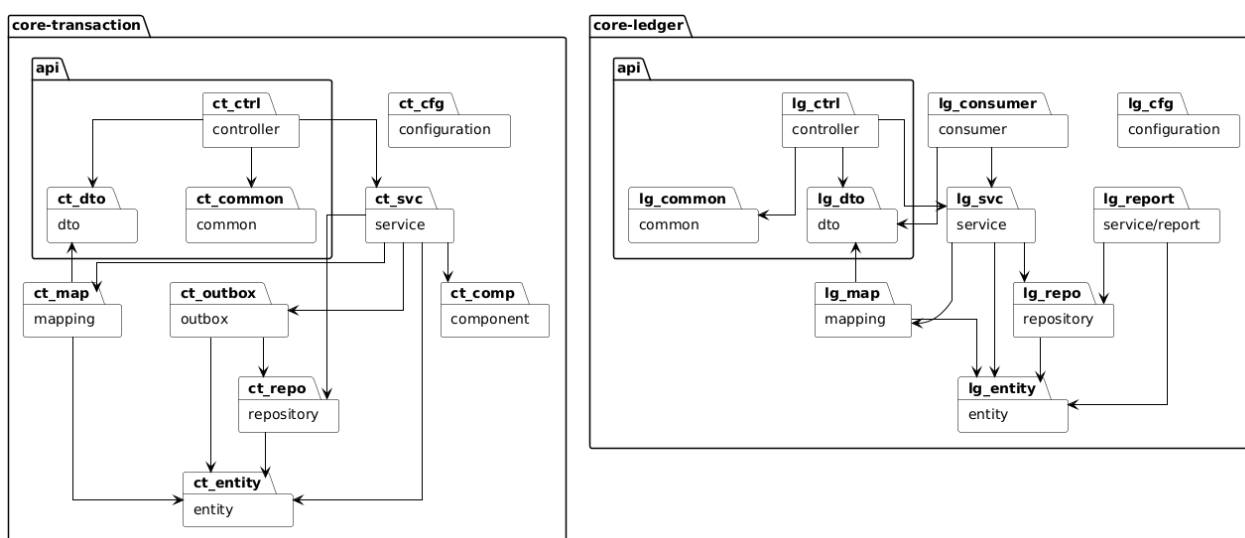


Рисунок 4 – Структурная схема ПО

На диаграмме отражена структура обоих сервисов. В **core-transaction** контроллеры принимают запросы и передают управление в слой сервисов, который содержит бизнес-логику и взаимодействует с репозиториями, маппингом и Outbox-компонентом для публикации событий. В **core-ledger** аналогичная слоистая структура дополнена пакетом **consumer** (Kafka-потребитель), который вызывает сервис для обработки событий, и пакетом **service/report** для генерации отчётов. Направление зависимостей строго сверху вниз: контроллеры зависят от сервисов, сервисы — от репозитория и **entity**, нижние слои не зависят от верхних.

## **6.2 Построение архитектуры информационной системы**

Архитектура информационной системы основывается на принципе разделения ответственности (CQRS). Сервис core-transaction выполняет роль стороны записи: REST API принимает команды, сервисный слой реализует бизнес-логику транзакций, репозитории обеспечивают доступ к данным, а компонент Outbox гарантирует доставку событий в Kafka. Сервис core-ledger выполняет роль стороны чтения: Kafka-потребитель получает события и сохраняет их в денормализованную модель, а REST API предоставляет историю транзакций и генерирует отчёты.

Подробное отображение компонентов и зависимостей представлено далее в диаграмме компонентов (п. 6.4).

Общая архитектура обеспечивает сопровождаемость: изменения в правилах обработки транзакций затрагивают только сервисный слой core-transaction, а изменения в логике запросов истории — только core-ledger.

## **6.3 Разработка функциональной модели**

Диаграмма деятельности в UML отображает последовательность действий, решения (ветвления), параллельные потоки и переходы. Выбран основной сценарий работы системы — проведение финансовой операции от момента запроса до появления записи в реестре.

**Функциональная модель: проведение транзакции**

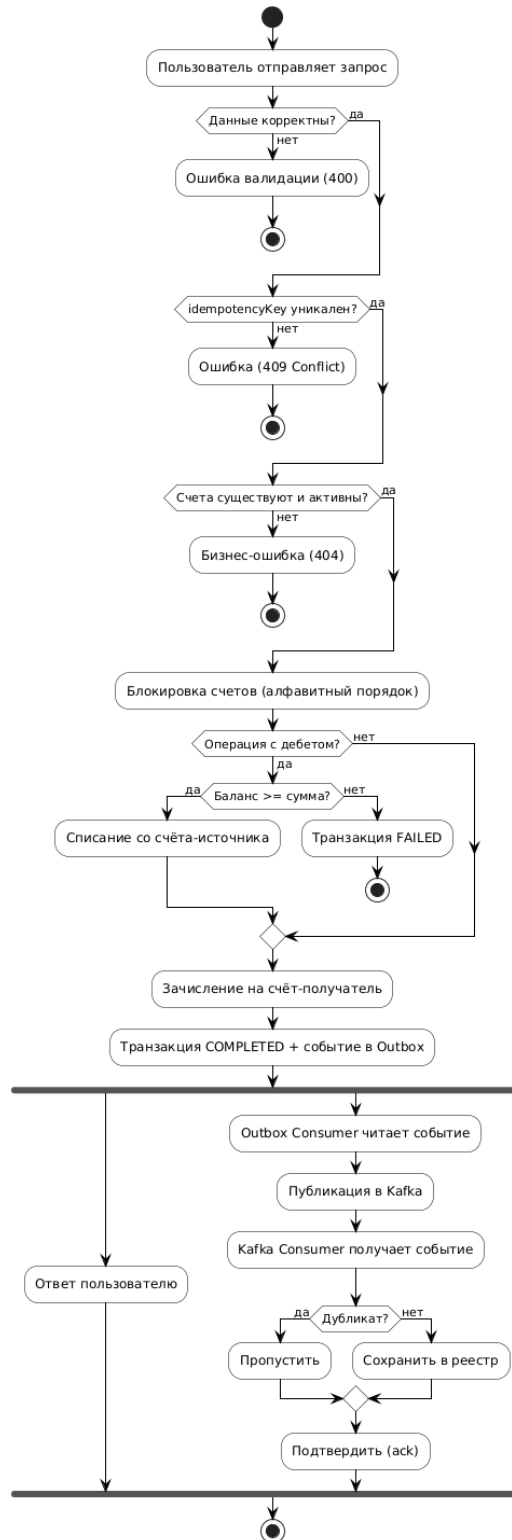


Рисунок 5 – Функциональная модель

На диаграмме показан полный путь транзакции. Сервис core-transaction последовательно проверяет входные данные, ключ идемпотентности, существование и активность счетов, достаточность баланса. При успехе



выполняется списание, зачисление, атомарная запись транзакции и события. Далее потоки расходятся: ответ возвращается пользователю немедленно, а Outbox Consumer асинхронно публикует событие в Kafka, откуда core-ledger получает его, проверяет на дубликат и сохраняет в реестр. Для операций без списания (денежный подарок, компенсация) ветвь проверки баланса пропускается.

## 6.4 Диаграмма компонентов

Диаграмма компонентов отображает физические или логические части системы, их интерфейсы и зависимости. На диаграмме выделены основные компоненты обоих сервисов и внешние системы.

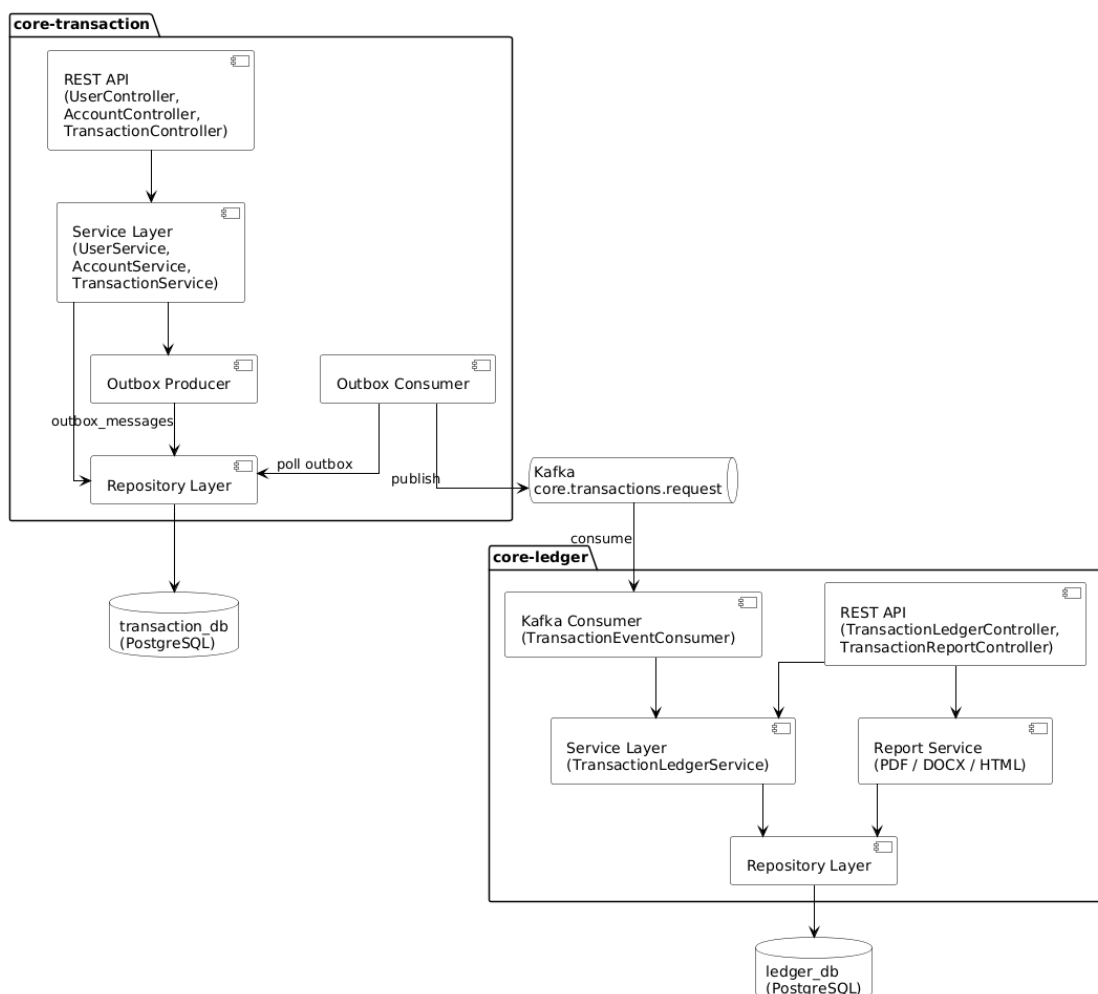


Рисунок 6 – Диаграмма компонентов

Зависимости между компонентами отражают поток данных: клиент обращается к REST API сервиса core-transaction; API использует сервисный слой; сервисный слой вызывает репозитории и Outbox Producer, который сохраняет событие в базу; Outbox Consumer забирает события и публикует их в Kafka; на стороне core-ledger Kafka Consumer получает сообщение и передаёт его в сервисный слой для записи в реестр. REST API сервиса core-ledger обращается к сервисному слою и к Report Service для формирования отчётов.

## 6.5 Диаграммы кооперации

Для сценария «Перевод между счетами» построена диаграмма кооперации. Она отражает последовательность взаимодействия между участниками в виде связей между объектами.

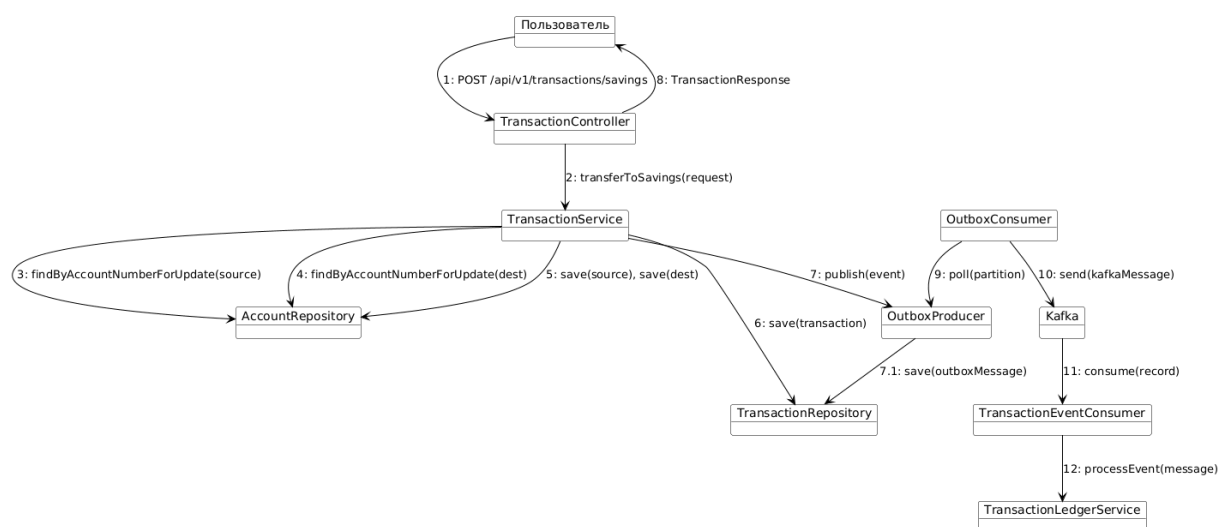


Рисунок 7 – Диаграмма кооперации

На диаграмме пользователь инициирует перевод через контроллер (1), контроллер делегирует сервису (2), сервис блокирует счета (3–4), изменяет балансы (5), сохраняет транзакцию (6) и публикует событие в Outbox (7) — всё в рамках одной БД-транзакции. Ответ возвращается пользователю (8). Асинхронно Outbox Consumer забирает событие (9) и публикует его в Kafka (10), после чего core-ledger получает сообщение (11) и записывает его в реестр (12).

## 6.6 Диаграмма размещения

Диаграмма размещения показывает физическое расположение программных компонентов по узлам инфраструктуры. Выделены основные узлы: клиентское устройство, два сервиса приложений, кластер Kafka и две базы данных.

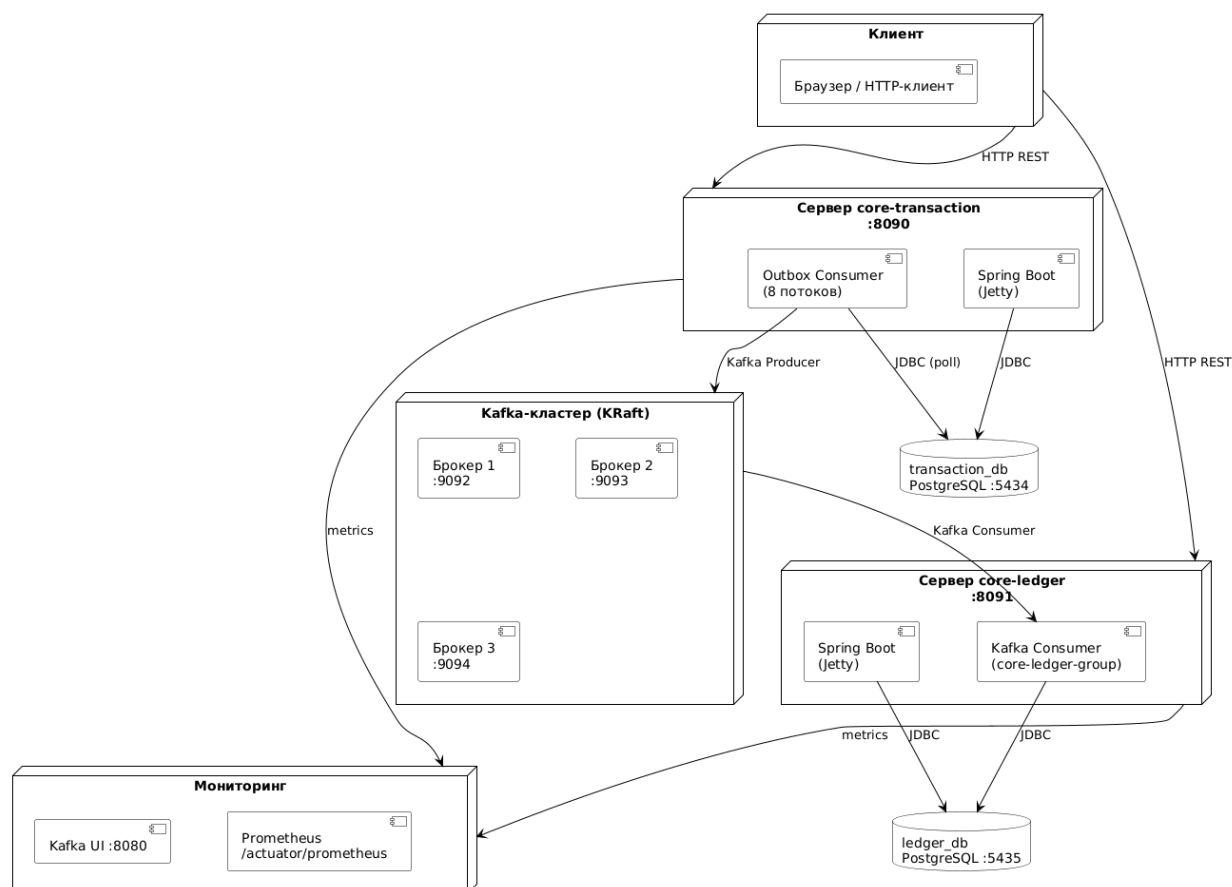


Рисунок 8 – Диаграмма размещения

Клиент обращается к двум сервисам по HTTP. core-transaction работает на порту 8090, взаимодействует с transaction\_db через JDBC и публикует события в Kafka-кластер из трёх KRaft-брокеров. core-ledger на порту 8091 потребляет события из Kafka и сохраняет записи в ledger\_db. Каждый сервис экспонирует метрики Prometheus на /actuator/prometheus, а Kafka UI на порту 8080 предоставляет веб-интерфейс для мониторинга топиков и потребителей.

## 6.7 Диаграмма состояний

Диаграмма состояний описывает жизненный цикл финансовой транзакции, outbox события, записи реестра — ключевых сущностей предметной области.

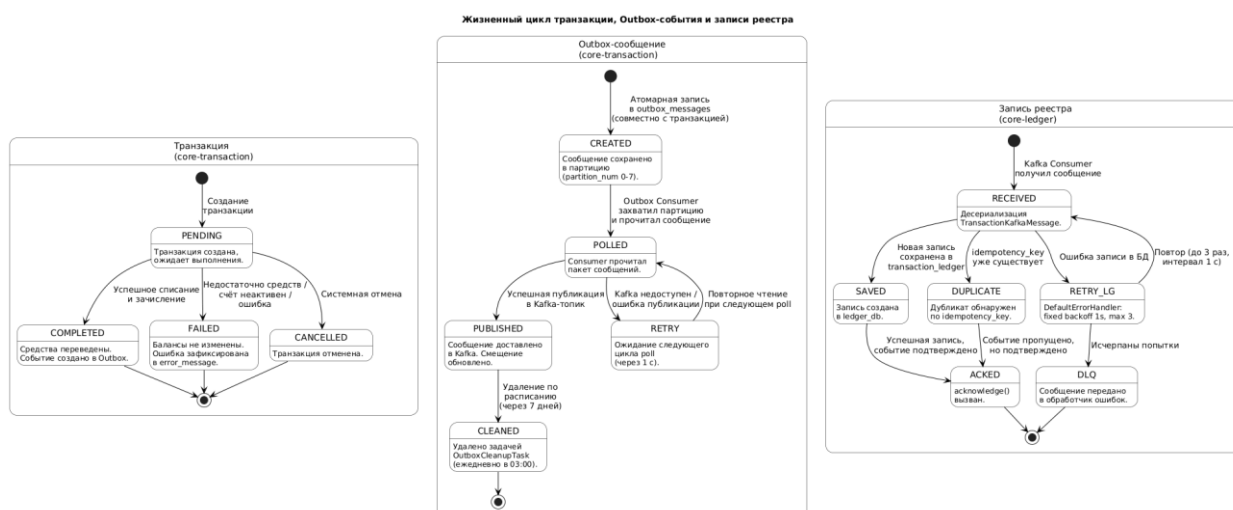


Рисунок 9 – Диаграмма состояний

На диаграмме отражены три жизненных цикла. Транзакция создаётся в состоянии PENDING и переходит в COMPLETED, FAILED или CANCELLED. При статусе COMPLETED атомарно создаётся Outbox-сообщение, которое проходит путь от создания через чтение и публикацию до удаления по расписанию; при ошибке публикации сообщение остаётся в базе и повторно обрабатывается при следующем цикле poll. На стороне core-ledger полученное из Kafka событие проверяется на дубликат, сохраняется в реестр и подтверждается; при ошибке записи применяется политика повторных попыток (до 3 раз с интервалом 1 секунда).

## 6.8 Диаграммы потоков данных

Диаграмма потоков данных показывает перемещение данных и полезных нагрузок между компонентами при проведении транзакции.

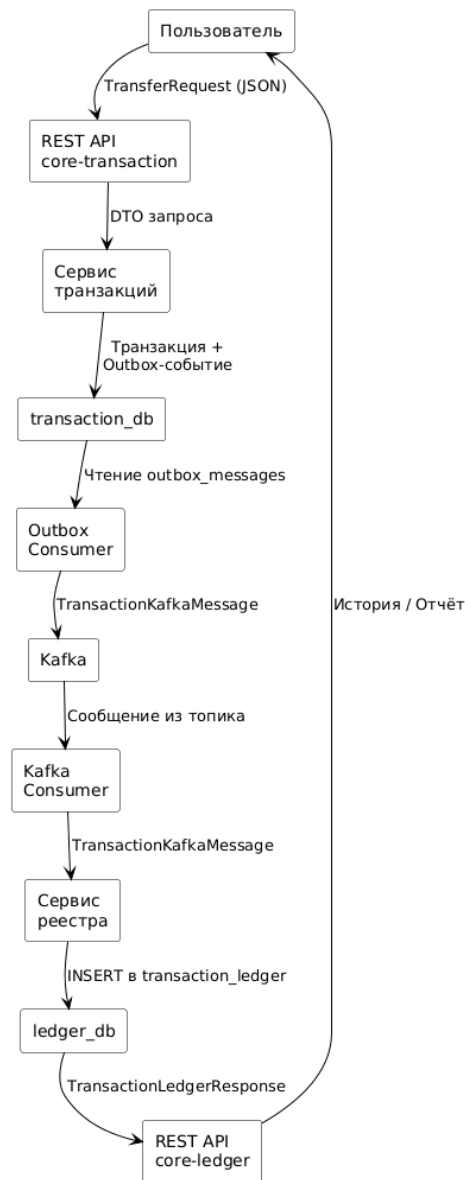


Рисунок 10 – Диаграмма потоков данных

Данные входят в систему как JSON-запрос, обрабатываются сервисом транзакций и фиксируются в transaction\_db вместе с Outbox-событием. Outbox Consumer публикует событие в Kafka, core-ledger получает его и сохраняет в реестр. Пользователь запрашивает историю или отчёт через REST API core-ledger.

## 6.9 Диаграмма классов

Диаграмма классов построена для каждого из двух сервисов. Отображены ключевые классы, интерфейсы, атрибуты и методы, а также связи наследования и ассоциации.

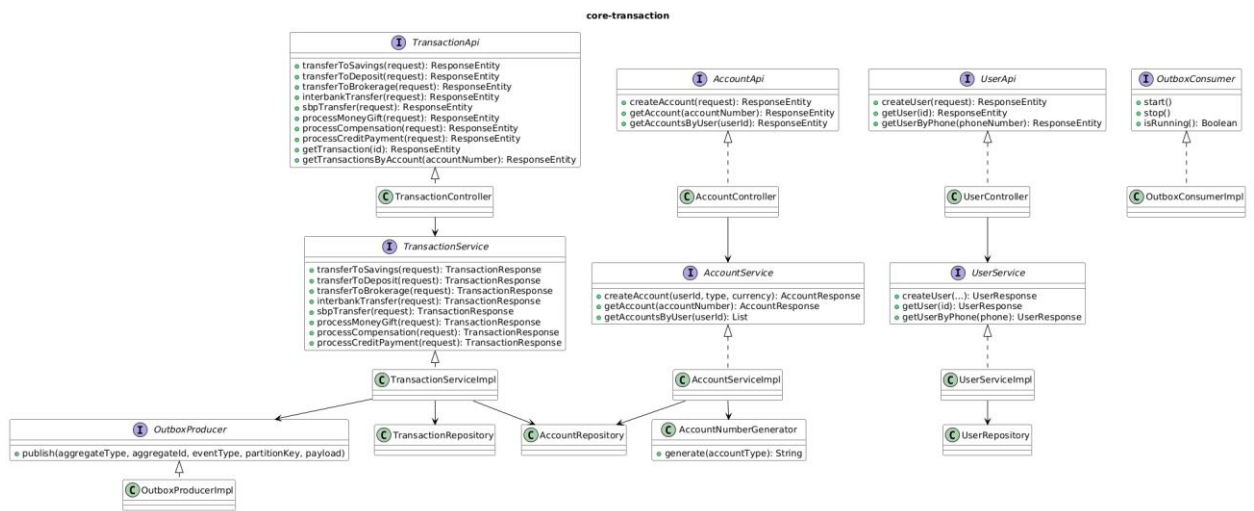


Рисунок 11 – Диаграмма классов сервиса core-transaction

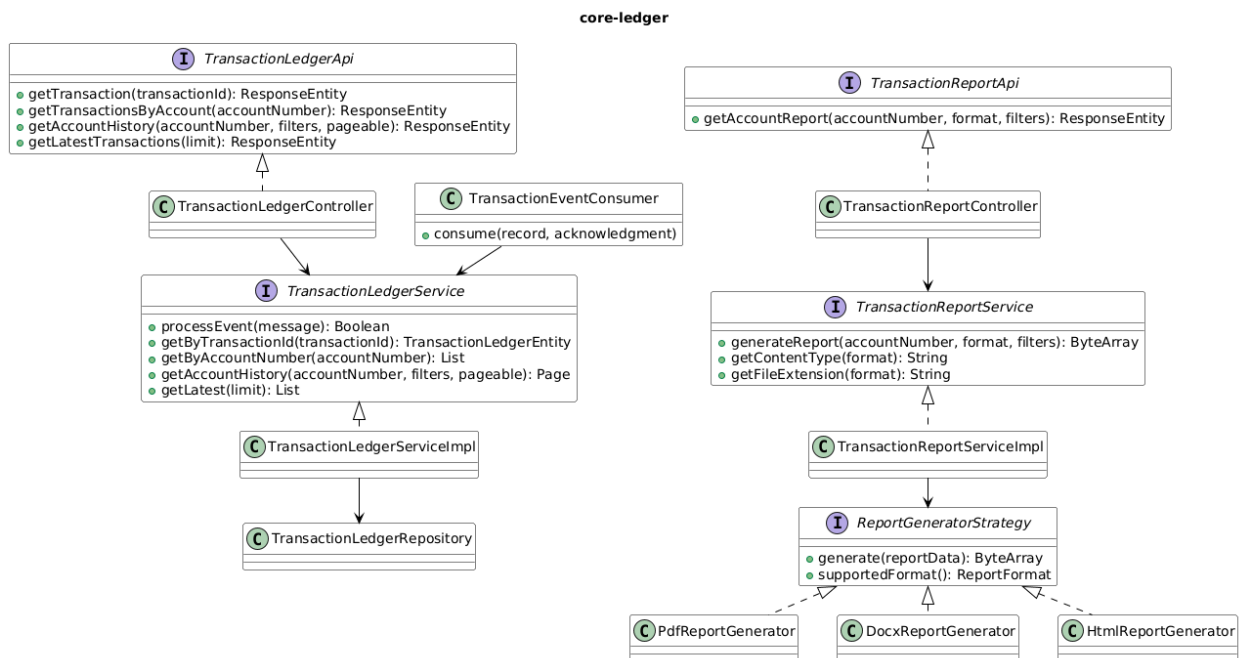


Рисунок 12 – Диаграмма классов сервиса core-ledger

На диаграммах классов видна чёткая структура обоих сервисов: контроллеры реализуют Api-интерфейсы с OpenAPI-аннотациями, сервисные классы реализуют бизнес-интерфейсы. В core-transaction ключевые связи: TransactionServiceImpl зависит от репозитория счетов и транзакций, а также от OutboxProducer. В core-ledger стратегия генерации отчётов реализована через паттерн Strategy: TransactionReportServiceImpl вызывает конкретный генератор (PdfReportGenerator, DocxReportGenerator, HtmlReportGenerator) в зависимости от запрошенного формата.

## 6.10 Диаграмма последовательности

Для сценария «Перевод на сберегательный счёт» построена диаграмма последовательности с синхронными вызовами и возвратами.

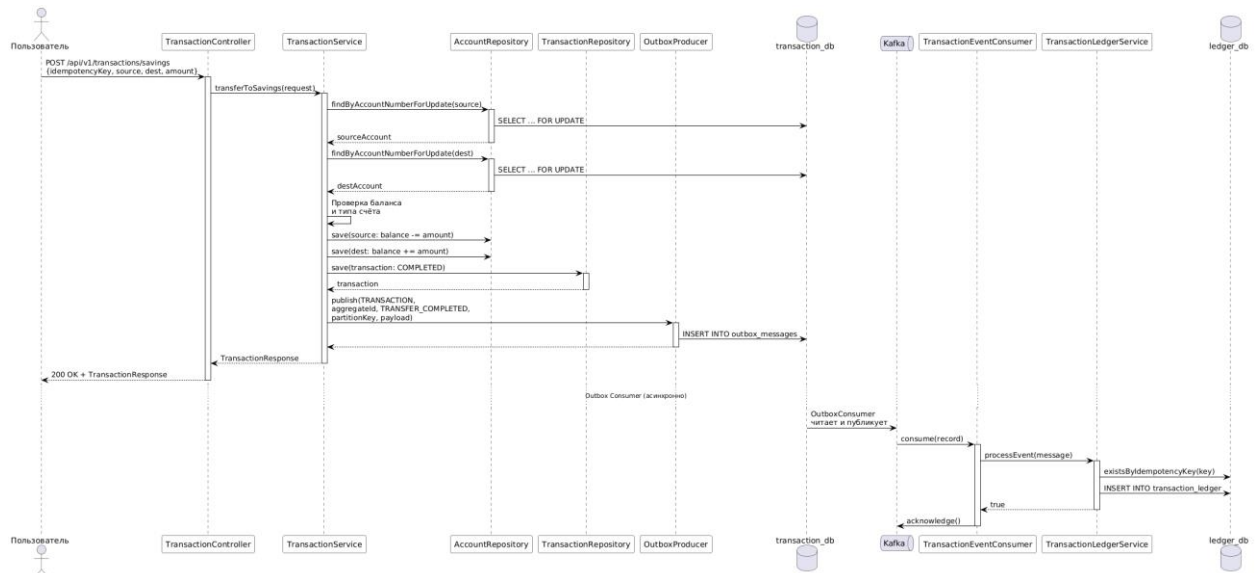


Рисунок 13 – Диаграмма последовательности

Диаграмма показывает синхронный сценарий перевода: контроллер делегирует сервису, сервис в рамках одной БД-транзакции блокирует счета, изменяет балансы, создаёт запись транзакции и сохраняет событие в Outbox. Ответ возвращается пользователю сразу после фиксации транзакции. Асинхронно Outbox Consumer забирает событие и публикует его в Kafka, после чего core-ledger получает сообщение, проверяет дубликат и сохраняет запись в реестр.

## 6.11 Диаграмма прецедентов

Диаграмма прецедентов отображает функциональность системы с точки зрения пользователей (актёров). Выделены основные прецеденты взаимодействия и связи «включает» (include) и «расширяет» (extend).

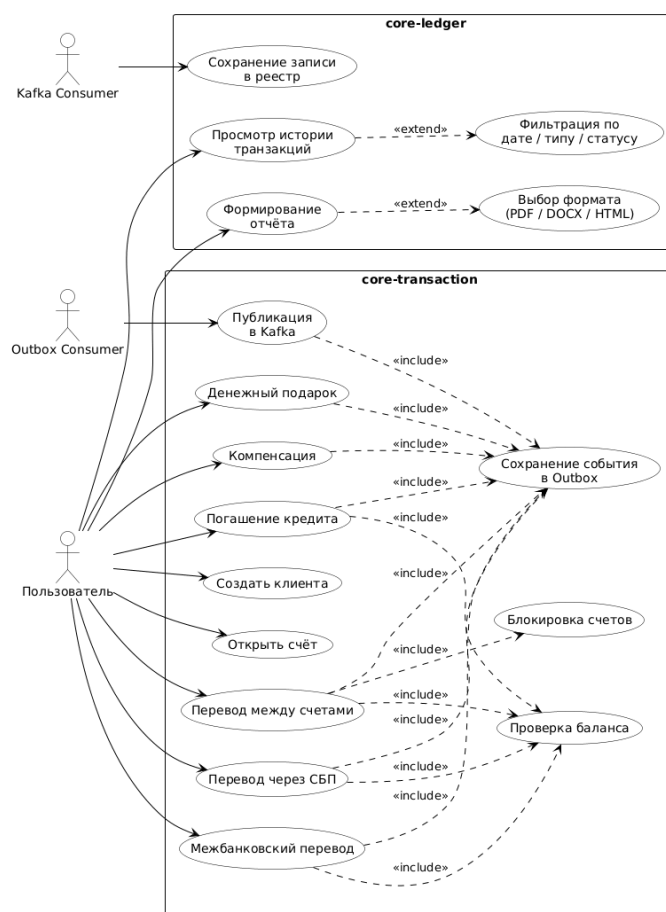


Рисунок 14 – Диаграмма прецедентов

Пользователь может выполнять все основные операции: создание клиента, открытие счёта, проведение шести типов транзакций, просмотр истории и формирование отчётов. Операции с дебетом (перевод, СБП, межбанковский, погашение кредита) включают проверку баланса. Все транзакции включают сохранение события в Outbox. Outbox Consumer отвечает за публикацию событий в Kafka. Kafka Consumer на стороне core-ledger сохраняет запись в реестр. Просмотр истории может быть расширен фильтрацией, а формирование отчёта — выбором формата.



## **7 Описание алгоритмов обработки данных**

### **7.1 Описание алгоритмов обработки данных**

В системе выделены четыре ключевых алгоритма, которые определяют поведение платформы при обработке финансовых операций и доставке событий между сервисами.

Алгоритм обработки финансовой транзакции (дебет-кредит). При поступлении запроса на перевод сервис `core-transaction` выполняет следующую последовательность действий в рамках одной БД-транзакции. Сначала проверяется ключ идемпотентности: если транзакция с таким ключом уже существует, возвращается ранее сохранённый результат без повторного выполнения. Затем выполняется поиск и блокировка счетов. Блокировка всегда накладывается в алфавитном порядке по номеру счёта — это исключает взаимную блокировку (deadlock) при одновременных встречных переводах. После блокировки проверяется активность счёта-источника и достаточность средств. Если проверки пройдены, выполняется списание со счёта-источника, зачисление на счёт-получатель, создание записи транзакции со статусом `COMPLETED` и атомарное сохранение события в таблицу `outbox_messages`. Для операций без списания (денежный подарок, компенсация) этап проверки баланса и списания пропускается — блокируется и пополняется только счёт-получатель.

Алгоритм работы `Outbox Consumer`. При запуске сервиса для каждой из восьми партиций создаётся отдельный поток, который с интервалом в 1 секунду выполняет цикл опроса. На каждом цикле поток пытается захватить лизинговую блокировку своей партиции (запись в таблице `outbox_partition_locks` с временем истечения). Если блокировка захвачена, поток считывает текущее смещение из `outbox_consumer_offsets`, загружает пакет сообщений (до 50 штук) из `outbox_messages` начиная с этого смещения, последовательно передаёт каждое сообщение обработчику (`TransactionEventHandler`), который публикует его в Kafka-топик. После

успешной обработки пакета смещение обновляется. Если обработка одного из сообщений завершается ошибкой, цикл прерывается — смещение обновляется до последнего успешно обработанного сообщения, а оставшиеся будут обработаны при следующем цикле.

Алгоритм идемпотентного потребления Kafka-событий. Kafka Consumer в сервисе core-ledger работает в режиме ручного подтверждения (AckMode.MANUAL\_IMMEDIATE). При получении сообщения из топика сервис извлекает idempotencyKey и проверяет его наличие в таблице transaction\_ledger. Если запись с таким ключом уже существует — событие считается дубликатом: оно пропускается, но подтверждается (acknowledge), чтобы Kafka не доставляла его повторно. Если ключ не найден — из сообщения создаётся новая запись реестра и сохраняется в базу. После успешного сохранения вызывается acknowledge. При ошибке записи срабатывает DefaultErrorHandler с политикой повторных попыток (до 3 раз с интервалом 1 секунда).

Алгоритм генерации номеров счетов. При создании счёта компонент AccountNumberGenerator формирует уникальный 20-символьный номер. Сначала определяется префикс типа: CHECKING → «1», SAVINGS → «2», DEPOSIT → «3», BROKERAGE → «4». Затем вызывается nextval() соответствующей PostgreSQL-последовательности (seq\_account\_checking, seq\_account\_savings и т.д.) через JdbcTemplate. Полученное значение дополняется нулями слева до 19 символов и конкатенируется с префиксом. Использование отдельных последовательностей для каждого типа гарантирует уникальность без конкуренции между типами.

## **7.2 Блок-схемы алгоритмов обработки данных**

Алгоритм обработки финансовой транзакции (дебет-кредит):

### Алгоритм обработки транзакции (дебет-кредит)

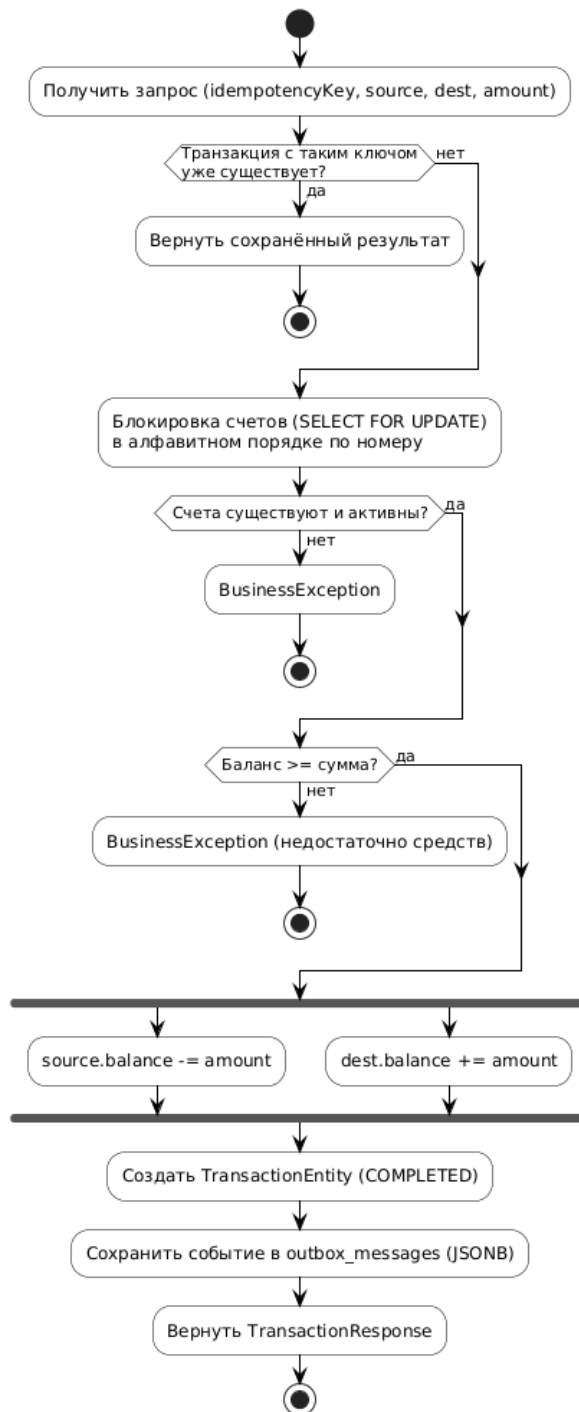


Рисунок 15 – Блок-схема алгоритма обработки транзакции  
Алгоритм работы Outbox Consumer (одна партиция):

**Алгоритм Outbox Consumer (одна партиция)**

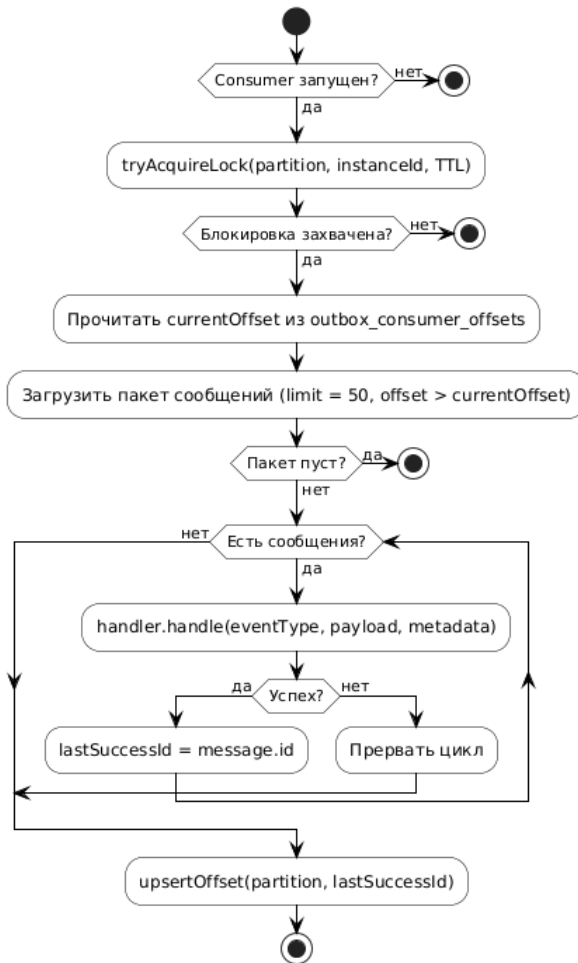


Рисунок 16 – Блок-схема алгоритма Outbox Consumer  
Алгоритм идемпотентного потребления Kafka-событий:

**Алгоритм потребления Kafka-событий (core-ledger)**

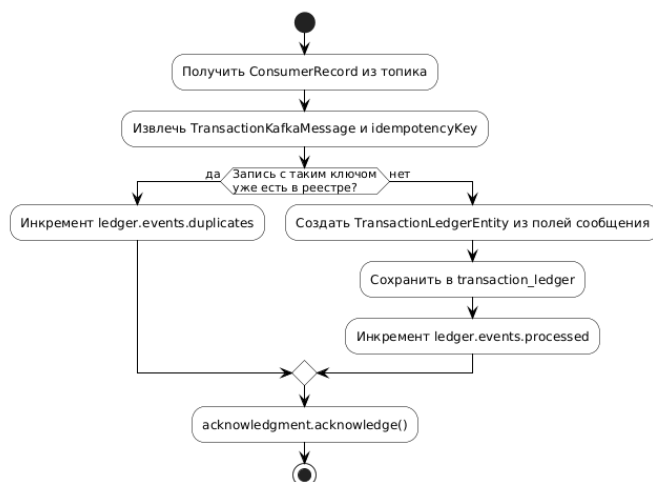


Рисунок 17 — Блок-схема алгоритма потребления Kafka-событий

## Алгоритм генерации номера счёта:

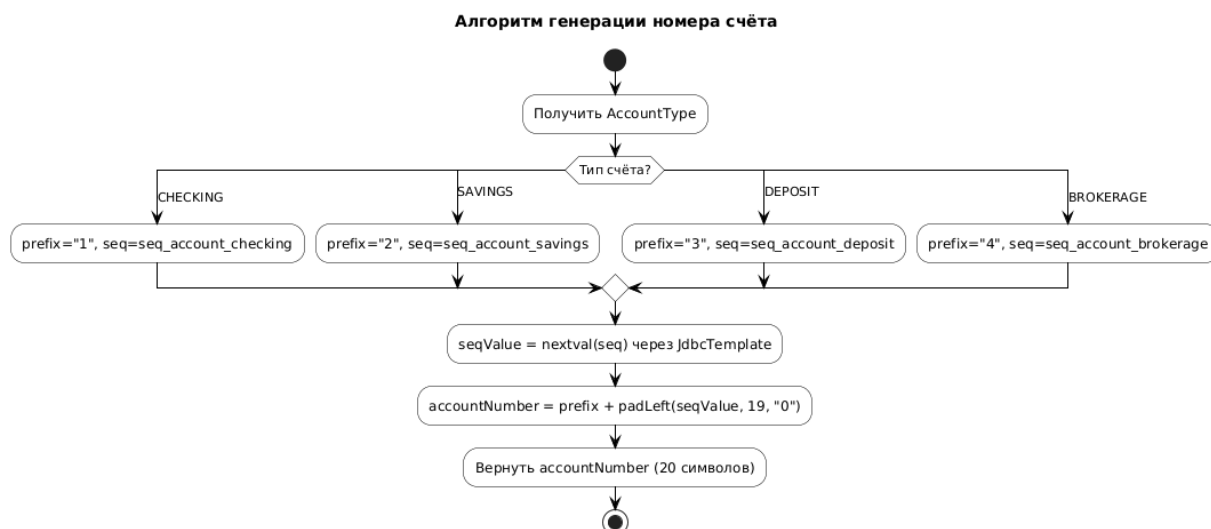


Рисунок 18 — Блок-схема алгоритма генерации номера счёта

На блок-схемах отражены все ключевые ветвления и последовательности действий. Алгоритм обработки транзакции показывает проверку идемпотентности, упорядоченную блокировку счетов, валидацию и атомарное сохранение данных вместе с Outbox-событием. Алгоритм Outbox Consumer демонстрирует лизинговую блокировку, пакетную обработку и обновление смещения. Алгоритм потребления Kafka-событий иллюстрирует идемпотентную дедупликацию по ключу и ручное подтверждение. Алгоритм генерации номера счёта показывает выбор последовательности по типу и формирование 20-символьного идентификатора.

## 8 Описание реализации программного продукта

### 8.1 Выбор среды программирования

Реализация системы выполнена на языке Kotlin 1.9.25 с целевой платформой Java 21. Kotlin выбран как основной язык разработки благодаря лаконичному синтаксису, встроенной null-безопасности и полной совместимости с экосистемой Java и Spring.

В качестве фреймворка используется Spring Boot 3.5.12. Встроенный сервер Tomcat заменён на Jetty — он легче по потреблению памяти и лучше подходит для микросервисной архитектуры. Управление зависимостями осуществляется через Gradle (Kotlin DSL).

Таблица 6 - Основные зависимости сервисов

| Компонент    | Зависимость                    | Назначение                      |
|--------------|--------------------------------|---------------------------------|
| ORM          | Spring Data JPA (Hibernate)    | Доступ к данным, репозитории    |
| Миграции     | Liquibase                      | Версионирование схемы БД        |
| Валидация    | spring-boot-starter-validation | Jakarta Bean Validation         |
| Сериализация | jackson-module-kotlin          | JSON (де)сериализация           |
| Логирование  | kotlin-logging-jvm 3.0.5       | Структурированное логирование   |
| Документация | springdoc-openapi 2.8.6        | Swagger UI на /swagger-ui.html  |
| Метрики      | Micrometer + Prometheus        | Экспорт на /actuator/prometheus |
| Сообщения    | Spring Kafka                   | Kafka Producer / Consumer       |

Сервис core-ledger дополнительно использует OpenPDF 2.0.3 для генерации PDF-отчётов и Apache POI 5.3.0 для генерации DOCX-отчётов.

Для тестирования используются JUnit 5, MockK 1.13.12 (мокирование для Kotlin), Testcontainers (PostgreSQL и Kafka в Docker-контейнерах) и Spring Kafka Test.

### 8.2 Последовательность разработки прикладной программы

Разработка выполнялась итеративно с привязкой к функциональным блокам:

- Проектирование схемы базы данных `transaction_db` и написание Liquibase-миграций: пользователи, счета, карты, транзакции, последовательности для генерации номеров.
- Реализация сервиса `core-transaction`: REST API для управления пользователями и счетами, бизнес-логика проведения транзакций с пессимистичной блокировкой.
- Реализация Transactional Outbox: таблицы `outbox_messages` (с LIST-секционированием), `outbox_consumer_offsets`, `outbox_partition_locks`; компоненты `OutboxProducer`, `OutboxConsumer`, `TransactionEventHandler`.
- Настройка Kafka-кластера (3 брокера KRaft) и конфигурация `Producer` в `core-transaction`.
- Проектирование схемы `ledger_db` и реализация сервиса `core-ledger`: Kafka Consumer с ручным подтверждением, идемпотентная запись в реестр, REST API для запроса истории.
- Реализация генерации отчётов в форматах PDF, DOCX и HTML через паттерн `Strategy`.
- Написание unit-тестов (`MockK`) и интеграционных тестов (`Testcontainers`).
- Оформление документации.

### 8.3 Схема меню приложения

Система не имеет графического пользовательского интерфейса в традиционном понимании — оба сервиса предоставляют REST API. В качестве интерактивного интерфейса для работы с API используется Swagger UI, доступный по адресу `/swagger-ui.html` на каждом из сервисов. Swagger UI генерируется автоматически библиотекой `springdoc-openapi` на основе аннотаций контроллеров.

Структура меню Swagger UI сервиса `core-transaction` (порт 8090) включает три группы эндпоинтов:

- user-controller — создание клиента, получение по идентификатору и по номеру телефона;
- account-controller — открытие счёта, получение по номеру счёта, получение всех счетов клиента;
- transaction-controller — проведение восьми типов транзакций, получение транзакции по идентификатору, получение транзакций по номеру счёта.

Структура меню Swagger UI сервиса core-ledger (порт 8091) включает две группы:

- transaction-ledger-controller — запрос записи по идентификатору транзакции, по номеру счёта, постраничная история с фильтрами, последние N транзакций;
- transaction-report-controller — формирование отчёта с выбором формата (PDF, DOCX, HTML).

С точки зрения пользователя ключевые сценарии начинаются с операций в core-transaction (создание клиента → открытие счёта → проведение транзакции), после чего результат можно просмотреть через core-ledger (история → отчёт).

## 8.4 Описание окон приложения

Каждая группа в Swagger UI раскрывается в список эндпоинтов. При выборе конкретного эндпоинта отображается его описание, схема запроса с типами полей и кнопка «Try it out» для интерактивного вызова.

Окно создания клиента (POST /api/v1/users). Содержит форму с полями: firstName (строка, обязательное), lastName (строка, обязательное), patronymic (строка, необязательное), phoneNumber (строка, формат +XXXXXXXXXX), email (строка, необязательное). При успешном создании возвращается JSON с присвоенным идентификатором и датой регистрации.



Окно открытия счёта (POST /api/v1/accounts). Содержит поля: `userId` (число), `accountType` (строка: CHECKING, SAVINGS, DEPOSIT, BROKERAGE), `currency` (строка, по умолчанию RUB). В ответе возвращается сгенерированный номер счёта, тип, баланс и статус.

Окно проведения транзакции (POST /api/v1/transactions/\*). Форма зависит от типа операции. Для переводов между счетами — `sourceAccountNumber`, `destinationAccountNumber`, `amount`, `idempotencyKey`. Для СБП — `sourceAccountNumber`, `recipientPhoneNumber`, `amount`. Для межбанковского перевода — `sourceCardNumber`, `destinationCardNumber`. При успешном выполнении возвращается тип, статус, сумма и временные метки.

Окно истории транзакций (GET /api/v1/transactions/account/{accountNumber}/history). Параметры фильтрации: `dateFrom`, `dateTo`, `type`, `status`, а также `page`, `size`, `sort`. Ответ содержит страничный список записей реестра.

Окно формирования отчёта (GET /api/v1/transactions/account/{accountNumber}/report). Параметр `format` задаёт формат выгрузки. При выполнении браузер скачивает файл с соответствующим расширением.

## 8.5 Функциональные возможности программного приложения

Функциональные возможности приложения соответствуют утверждённым требованиям:

- создание клиента банка с указанием ФИО, номера телефона и электронной почты;
- открытие счёта выбранного типа с автоматической генерацией уникального 20-символьного номера;
- проведение внутренних переводов между счетами (на сберегательный, депозитный, брокерский);
- проведение перевода через СБП по номеру телефона получателя;

- проведение межбанковского перевода по номерам платёжных карт;
- зачисление средств без списания: денежный подарок и компенсация;
- погашение кредита: списание с расчётного счёта и зачисление на счёт назначения;
- гарантированная доставка событий через Transactional Outbox и Kafka;
- идемпотентная обработка повторных запросов и повторно доставленных событий;
- запрос истории транзакций с фильтрацией по дате, типу операции и статусу;
- формирование отчётов по истории транзакций в форматах PDF, DOCX и HTML;
- экспорт метрик в формате Prometheus для мониторинга.

Перечисленный функционал полностью соответствует утверждённым требованиям, обеспечивая полный жизненный цикл банковской операции — от регистрации клиента до выгрузки отчёта.

## 8.6 Функции и процедуры обработки данных

Обработка данных выполняется на нескольких уровнях системы.

На уровне контроллеров REST API запросы принимаются типизированными методами. TransactionController делегирует все операции в TransactionService, принимая DTO (TransferRequest, SbpTransferRequest, InterbankTransferRequest, MoneyGiftRequest, CompensationRequest, CreditPaymentRequest) и возвращая TransactionResponse. TransactionLedgerController делегирует запросы истории в TransactionLedgerService и преобразует entity в DTO через маппер. TransactionReportController делегирует генерацию отчётов в

TransactionReportService, устанавливая Content-Type и Content-Disposition в заголовках ответа.

На уровне сервисов реализована основная бизнес-логика:

- метод `executeDebitCredit()` выполняет полный цикл перевода: списание, зачисление, создание записи транзакции и публикация события в Outbox — всё в одной аннотированной `@Transactional` операции;
- метод `executeCreditOnly()` аналогичен, но без списания — для денежных подарков и компенсаций;
- метод `lockAccountsInOrder()` упорядочивает счета по алфавиту и блокирует их через `SELECT ... FOR UPDATE`;
- метод `processEvent()` в сервисе реестра реализует идемпотентную запись: проверка существования ключа, создание entity, сохранение;
- метод `generateReport()` загружает историю, формирует объект `ReportData` и делегирует нужной стратегии генерации.

На уровне интеграции с Kafka обработка данных реализована в двух точках:

- `TransactionEventHandler.handle()` — десериализация Outbox-события, формирование `TransactionKafkaMessage` с ключом идемпотентности `"${aggregateId}-${messageId}"` и синхронная отправка в Kafka;
- `TransactionEventConsumer.consume()` — получение сообщения, вызов сервиса реестра, инкремент метрик и ручное подтверждение.

Обработка данных разделена по уровням: контроллеры обеспечивают приём и валидацию, сервисный слой реализует бизнес-правила с гарантией атомарности, а интеграционный слой обеспечивает надёжную доставку событий и их идемпотентное потребление.

## 9 Тестирование программного продукта и отладка

### 9.1 Стратегия тестирования

При разработке банковской платформы я уделил особое внимание тестированию, поскольку система работает с финансовыми операциями, где любая ошибка может привести к некорректному движению денежных средств. Для обеспечения качества я применил многоуровневую стратегию тестирования, включающую модульные (unit) и интеграционные тесты.

Модульные тесты проверяют логику отдельных компонентов в изоляции — сервисов, контроллеров, мапперов. Для подмены зависимостей я использовал библиотеку MockK, которая хорошо интегрируется с Kotlin и позволяет создавать моки, стабы и верифицировать вызовы методов.

Интеграционные тесты проверяют взаимодействие компонентов с реальной инфраструктурой — базой данных PostgreSQL и брокером сообщений Apache Kafka. Для этого я использовал библиотеку Testcontainers, которая автоматически поднимает Docker-контейнеры с необходимыми сервисами перед запуском тестов и останавливает их после завершения.

Всего в проекте написано 33 тестовых файла: 21 в сервисе core-transaction и 12 в сервисе core-ledger. Тесты организованы с помощью JUnit 5 и его механизма @Nested-классов, который позволяет группировать тест-кейсы по бизнес-операциям.

### 9.2 Инструменты тестирования

В таблице 9.1 представлены основные инструменты, использованные для тестирования.

Таблица 7 — Инструменты тестирования

| Инструмент | Назначение                                                                      |
|------------|---------------------------------------------------------------------------------|
| JUnit 5    | Фреймворк для запуска тестов, поддержка аннотаций @Test, @Nested, @BeforeEach   |
| MockK      | Библиотека мокирования для Kotlin — создание моков, стабов, верификация вызовов |

| Инструмент       | Назначение                                                                                                                      |
|------------------|---------------------------------------------------------------------------------------------------------------------------------|
| Testcontainers   | Автоматический запуск Docker-контейнеров (PostgreSQL 16, Apache Kafka 3.7.0)                                                    |
| Spring Boot Test | Аннотации <code>@SpringBootTest</code> , <code>@WebMvcTest</code> , <code>@AutoConfigureMockMvc</code> для интеграции со Spring |
| MockMvc          | Выполнение HTTP-запросов к контроллерам без реального запуска сервера                                                           |

### 9.3 Инструменты тестирования

Для интеграционных тестов каждого сервиса я создал базовый абстрактный класс `IntegrationTestBase`, от которого наследуются все интеграционные тесты.

В сервисе `core-transaction` базовый класс поднимает контейнер PostgreSQL 16 и перед каждым тестом очищает все таблицы в порядке, учитывающем внешние ключи: сначала удаляются сообщения Outbox, затем смещения и блокировки потребителей, далее транзакции, карты, счета и, наконец, пользователи. Это гарантирует, что каждый тест начинается с чистого состояния базы данных.

В сервисе `core-ledger` базовый класс поднимает два контейнера — PostgreSQL 16 и Apache Kafka 3.7.0, поскольку этот сервис является потребителем сообщений Kafka. Перед каждым тестом очищается таблица `transaction_ledger`.

Подключение к тестовым контейнерам настраивается динамически через механизм `@DynamicPropertySource`, который подставляет URL, логин и пароль контейнера в конфигурацию Spring. При этом Liquibase-миграции автоматически применяются к тестовой базе, а Hibernate работает в режиме `validate` — точно так же, как в `production`-окружении. Это позволяет убедиться, что миграции корректны и схема базы данных соответствует маппингу сущностей.

## 9.4 Тестовые утилиты

Для удобства создания тестовых данных я реализовал вспомогательный объект `TestDataFactory` в каждом сервисе. Он содержит фабричные методы с параметрами по умолчанию для всех типов запросов и сущностей. Например, вызов `TestDataFactory.transferRequest()` создаёт готовый объект запроса на перевод с валидными данными, а при необходимости можно переопределить любое поле.

В сервисе `core-transaction` дополнительно создан компонент `TestApiClient`, который оборачивает `MockMvc` и предоставляет высокоуровневые методы для подготовки тестовых данных: создание пользователя, открытие счёта, выпуск карты, пополнение баланса. Это позволяет в интеграционных тестах писать сценарии на уровне бизнес-операций, а не HTTP-запросов.

## 9.5 Модульные тесты

Модульные тесты покрывают основную бизнес-логику каждого сервиса. Рассмотрим ключевые группы.

Сервис `core-transaction`:

- `TransactionServiceImplTest` — 25+ тестов для логики проведения транзакций: проверка идемпотентности, валидация счетов, списание и зачисление средств, обработка ошибок (недостаточно средств, несуществующий счёт, неверный тип счёта назначения);
- `AccountServiceImplTest` — тесты создания счетов, генерации номеров, получения информации по номеру счёта;
- `UserServiceImplTest` — тесты регистрации пользователей и проверки уникальности данных;
- `OutboxProducerImplTest` — тесты записи событий в Outbox-таблицу с корректным вычислением партии;
- `OutboxConsumerImplTest` — тесты опроса партиций, получения блокировок, пакетной обработки сообщений;

- TransactionControllerTest, AccountControllerTest, UserControllerTest — тесты REST-контроллеров через @WebMvcTest с проверкой валидации входных данных и формата ответов;

- MapperTest — тесты маппинга между сущностями и DTO.

Сервис core-ledger:

- TransactionLedgerServiceImplTest — тесты обработки событий с проверкой идемпотентности, построения фильтров для истории операций;

- TransactionEventConsumerTest — тесты десериализации Kafka-сообщений и передачи в сервисный слой;

- PdfReportGeneratorTest, DocxReportGeneratorTest, HtmlReportGeneratorTest — тесты генерации отчётов в каждом из поддерживаемых форматов;

- TransactionReportServiceImplTest — тесты стратегии выбора генератора отчётов по формату;

- TransactionLedgerMapperTest — тесты маппинга Kafka-сообщений в сущности леджера.

## 9.6 Интеграционные тесты

Интеграционные тесты проверяют полные сценарии работы системы с реальной базой данных и брокером сообщений.

Сервис core-transaction:

Главный интеграционный тест TransactionIntegrationTest содержит 9 вложенных классов, каждый из которых проверяет отдельный тип операции:

- TransferToSavings — перевод на сберегательный счёт: успешный перевод, проверка идемпотентности (повторный запрос с тем же ключом не создаёт дубль), недостаточно средств, неверный тип счёта назначения;

- TransferToDeposit — перевод на депозитный счёт;

- TransferToBrokerage — перевод на брокерский счёт;

- MoneyGift — денежный подарок (операция без списания);

- Compensation — компенсация (операция без списания);

- CreditPayment — платёж по кредиту;
- SbpTransfer — перевод через СБП по номеру телефона;
- InterbankTransfer — межбанковский перевод по номеру карты;
- GetTransaction — получение информации о транзакции.

Дополнительные интеграционные тесты:

- AccountIntegrationTest — создание счетов разных типов и проверка генерации уникальных номеров;
- AccountNumberGeneratorIntegrationTest — проверка работы PostgreSQL-последовательностей для генерации номеров;
- CardNumberGeneratorIntegrationTest — проверка генерации номеров карт;
- UserIntegrationTest — регистрация пользователей;
- OutboxProducerIntegrationTest — запись событий в партиционированную Outbox-таблицу;
- OutboxConsumerIntegrationTest — опрос партиций и обработка сообщений;
- TransactionEventHandlerIntegrationTest — отправка событий в Kafka.

Сервис core-ledger:

- TransactionLedgerIntegrationTest — тесты REST-запросов для получения транзакций: по идентификатору, по номеру счёта, история с фильтрацией по дате, типу и статусу, пагинация;
- KafkaConsumerIntegrationTest — полный цикл: отправка сообщения в Kafka-топик → асинхронное потребление → проверка записи в базу данных. Этот тест подтверждает корректность всей цепочки от получения события до его сохранения;
- TransactionReportIntegrationTest — генерация отчётов в форматах PDF, DOCX и HTML через REST API.



## 9.7 Запуск и результаты тестирования

Запуск всех тестов выполняется командой `./gradlew test` из директории соответствующего сервиса. Gradle автоматически компилирует код, поднимает Testcontainers и выполняет все тесты.

### Test Summary

|       |          |         |          |            |
|-------|----------|---------|----------|------------|
| 212   | 0        | 0       | 23.824s  | 100%       |
| tests | failures | ignored | duration | successful |

| Packages                                                 | Classes |       |          |         |          |              |
|----------------------------------------------------------|---------|-------|----------|---------|----------|--------------|
| Package                                                  |         | Tests | Failures | Ignored | Duration | Success rate |
| <a href="#">by.svyat.core.transaction.api.controller</a> |         | 29    | 0        | 0       | 2.771s   | 100%         |
| <a href="#">by.svyat.core.transaction.component</a>      |         | 3     | 0        | 0       | 0.332s   | 100%         |
| <a href="#">by.svyat.core.transaction.integration</a>    |         | 89    | 0        | 0       | 19.049s  | 100%         |
| <a href="#">by.svyat.core.transaction.mapping</a>        |         | 8     | 0        | 0       | 0.002s   | 100%         |
| <a href="#">by.svyat.core.transaction.outbox.impl</a>    |         | 42    | 0        | 0       | 0.679s   | 100%         |
| <a href="#">by.svyat.core.transaction.service.impl</a>   |         | 41    | 0        | 0       | 0.991s   | 100%         |

Рисунок 19 - Результат запуска тестов сервиса core-transaction

### Test Summary

|       |          |         |          |            |
|-------|----------|---------|----------|------------|
| 50    | 0        | 0       | 15.577s  | 100%       |
| tests | failures | ignored | duration | successful |

| Packages                                            | Classes |       |          |         |          |              |
|-----------------------------------------------------|---------|-------|----------|---------|----------|--------------|
| Package                                             |         | Tests | Failures | Ignored | Duration | Success rate |
| <a href="#">by.svyat.core.ledger.consumer</a>       |         | 2     | 0        | 0       | 0.697s   | 100%         |
| <a href="#">by.svyat.core.ledger.integration</a>    |         | 16    | 0        | 0       | 13.991s  | 100%         |
| <a href="#">by.svyat.core.ledger.mapping</a>        |         | 1     | 0        | 0       | 0.001s   | 100%         |
| <a href="#">by.svyat.core.ledger.service.impl</a>   |         | 18    | 0        | 0       | 0.575s   | 100%         |
| <a href="#">by.svyat.core.ledger.service.report</a> |         | 13    | 0        | 0       | 0.313s   | 100%         |

Рисунок 20 - Результат запуска тестов сервиса core-ledger

По результатам тестирования все тесты обоих сервисов завершились успешно. В сервисе core-transaction выполнен 21 тестовый класс, покрывающий все типы банковских операций — переводы между счетами, межбанковские переводы, переводы через СБП, денежные подарки, компенсации и платежи по кредитам. Интеграционные тесты подтвердили корректность работы с реальной базой данных PostgreSQL 16, включая генерацию уникальных номеров счетов и карт через последовательности, механизм идемпотентности, пессимистичную блокировку счетов и запись событий в Outbox-таблицу.

В сервисе core-ledger выполнено 12 тестовых классов. Интеграционные тесты подтвердили корректность полного цикла обработки событий: от получения сообщения из Kafka-топика до его сохранения в таблице transaction\_ledger с проверкой идемпотентности. Также проверена работа REST API для получения истории операций с фильтрацией и пагинацией, а также генерация отчётов в форматах PDF, DOCX и HTML.

Ошибок и сбоев в ходе тестирования выявлено не было. Все проверки — валидация входных данных, обработка граничных случаев (недостаточно средств, несуществующий счёт, дублирующий запрос), корректность балансов после операций — пройдены успешно, что подтверждает готовность системы к эксплуатации.

## 10 Проектирование технической и справочной информации к ПО

### 10.1 Требования к окружению и подключение

В рамках работы используется локальное окружение: оба микросервиса, PostgreSQL и Kafka предполагаются доступными на машине разработчика в Docker-контейнерах с пробросом портов на localhost. Промышленное или облачное развёртывание здесь не детализируется.

Для локального запуска нужны установленные Java 21 (JDK) и Docker с Docker Compose. Порядок запуска инфраструктуры:

- Kafka-кластер — из корневой директории проекта: `docker-compose up -d`. Поднимаются три KRaft-брокера (порты 9092, 9093, 9094) и Kafka UI (порт 8080).
- БД core-transaction — из директории core-transaction: `docker-compose up -d`. Поднимается PostgreSQL 16 на порту 5434, база `transaction_db`.
- БД core-ledger — из директории core-ledger: `docker-compose up -d`. Поднимается PostgreSQL 16 на порту 5435, база `ledger_db`.

После запуска инфраструктуры сервисы стартуют командой `./gradlew bootRun` из своих директорий. Liquibase автоматически применяет миграции при первом запуске.

Основные параметры сервиса core-transaction задаются в файле `core-transaction/src/main/resources/application.yaml`:

- `server.port=8090` — порт HTTP-сервера;
- `spring.datasource.url=jdbc:postgresql://localhost:5434/transaction_db` — подключение к PostgreSQL;
- `spring.kafka.bootstrap-servers=localhost:9092,localhost:9093,localhost:9094` — адреса Kafka-брокеров;
- `outbox.partition-count=8` — количество партиций Outbox-таблицы;
- `outbox.poll-interval-ms=1000` — интервал опроса партиций (мс);
- `outbox.batch-size=50` — размер пакета сообщений при опросе;

- `outbox.consumer-group=core-transaction` — имя группы потребителя;
- `outbox.lock-ttl-seconds=30` — время жизни lease-блокировки партии (с);
- `outbox.cleanup-retention-days=7` — срок хранения обработанных сообщений (дней).

Основные параметры сервиса `core-ledger` задаются в файле `core-ledger/src/main/resources/application.yaml`:

- `server.port=8091` — порт HTTP-сервера;
- `spring.datasource.url=jdbc:postgresql://localhost:5435/ledger_db` — подключение к PostgreSQL;
- `spring.kafka.bootstrap-servers=localhost:9092,localhost:9093,localhost:9094` — адреса Kafka-брокеров;
- `kafka.consumer.topic=core.transactions.request` — имя Kafka-топика для потребления;
- `kafka.consumer.group-id=core-ledger-group` — группа потребителей Kafka.

Параметры подключения к PostgreSQL (логин, пароль) задаются в тех же файлах через поля `spring.datasource.username` и `spring.datasource.password`. Метрики обоих сервисов доступны по пути `/actuator/prometheus`.

## 10.2 Основные REST endpoint

REST API реализовано в обоих сервисах и использует префикс `/api/v1`. Интерактивная документация доступна через Swagger UI: `http://localhost:8090/swagger-ui.html` (`core-transaction`) и `http://localhost:8091/swagger-ui.html` (`core-ledger`).

Endpoint сервиса `core-transaction` (запись):

- `POST /api/v1/users` — регистрация нового пользователя;

- GET /api/v1/users/{id} — получение пользователя по идентификатору;
- GET /api/v1/users/phone/{phoneNumber} — поиск пользователя по номеру телефона;
- POST /api/v1/accounts — открытие нового банковского счёта;
- GET /api/v1/accounts/{accountNumber} — получение счёта по номеру;
- GET /api/v1/accounts/user/{userId} — получение всех счетов пользователя;
- POST /api/v1/transactions/savings — перевод на сберегательный счёт;
- POST /api/v1/transactions/deposit — перевод на депозитный счёт;
- POST /api/v1/transactions/brokerage — перевод на брокерский счёт;
- POST /api/v1/transactions/interbank — межбанковский перевод по номеру карты;
- POST /api/v1/transactions/sbp — перевод через СБП по номеру телефона;
- POST /api/v1/transactions/gift — денежный подарок (зачисление без списания);
- POST /api/v1/transactions/compensation — компенсация (зачисление без списания);
- POST /api/v1/transactions/credit-payment — платёж по кредиту;
- GET /api/v1/transactions/{id} — получение транзакции по идентификатору;
- GET /api/v1/transactions/account/{accountNumber} — получение транзакций по номеру счёта.

Во всех POST-эндпоинтах транзакций в теле запроса передаётся ключ идемпотентности (`idempotencyKey`), который предотвращает повторное выполнение операции при дублирующем запросе.

Endpoint сервиса core-ledger (чтение):

- GET /api/v1/transactions/{transactionId} — получение записи по идентификатору исходной транзакции;
- GET /api/v1/transactions/account/{accountNumber} — все записи по номеру счёта;
- GET /api/v1/transactions/account/{accountNumber}/history — история операций с фильтрацией по дате, типу и статусу, с пагинацией;
- GET /api/v1/transactions/latest — последние N транзакций (по умолчанию 20);
- GET /api/v1/transactions/account/{accountNumber}/report — генерация отчёта в формате PDF, DOCX или HTML.

Ошибки возвращаются в едином формате ErrorResponse с полями timestamp, status, message и path

### 10.3 Формат сообщений для Kafka

При успешном завершении транзакции Outbox-потребитель публикует в топик core.transactions.request сообщение типа TransactionKafkaMessage, сериализованное в JSON.

Сообщение содержит следующие поля:

- idempotencyKey — ключ идемпотентности в формате "{aggregateId}-{messageId}" (String);
- eventType — тип события (например, TRANSFER\_COMPLETED);
- payload — тело события с данными транзакции;
- metadata — метаданные события.

Тело события (payload) содержит:

- transactionId — идентификатор исходной транзакции (Long);
- type — тип операции (TRANSFER\_SAVINGS, TRANSFER\_DEPOSIT, TRANSFER\_BROKERAGE, INTERBANK\_TRANSFER, SBP\_TRANSFER, MONEY\_GIFT, COMPENSATION, CREDIT\_PAYMENT);

- `sourceAccountNumber` — номер счёта списания (null для операций без списания);
- `destinationAccountNumber` — номер счёта зачисления;
- `amount` — сумма операции (BigDecimal);
- `currency` — валюта, по умолчанию RUB;
- `status` — статус транзакции (COMPLETED или FAILED).

Метаданные (metadata) содержат:

- `aggregateId` — идентификатор агрегата, используется как ключ Kafka-сообщения для гарантии порядка;
- `partitionKey` — ключ партиционирования (номер счёта);
- `sourceMessageId` — идентификатор исходного Outbox-сообщения;
- `createdAt` — время создания события в формате ISO 8601.

Consumer в `core-ledger` извлекает `idempotencyKey` и проверяет наличие записи с таким ключом в таблице `transaction_ledger`. Если запись существует — сообщение считается дубликатом и пропускается. Если нет — данные из `payload` и `metadata` мапятся в новую запись леджера и сохраняются в базу данных. После успешного сохранения вызывается `acknowledgment.acknowledge()`, что подтверждает сообщение на стороне Kafka.

## 10.4 Реализация паттерна Transactional Outbox

Для надёжной доставки событий между сервисами я реализовал паттерн Transactional Outbox. Вместо прямой отправки в Kafka событие записывается в таблицу `outbox_messages` в рамках той же транзакции базы данных, что и бизнес-операция. Если транзакция откатывается — откатывается и событие. Фоновый процесс (Outbox Consumer) периодически читает новые сообщения из таблицы и публикует их в Kafka.

Таблица `outbox_messages` использует LIST-партиционирование PostgreSQL по полю `partition_num` — создано 8 партиций (от `outbox_messages_p0` до `outbox_messages_p7`). Номер партиции вычисляется по

формуле `abs(partitionKey.hashCode()) % 8`, где `partitionKey` — номер счёта. Это гарантирует, что все события одного счёта обрабатываются последовательно в одной партии.

`Outbox Consumer` при старте приложения создаёт пул из 8 потоков — по одному на партию. Каждый поток с интервалом 1 секунда выполняет цикл: захват `lease`-блокировки (TTL 30 секунд) → чтение пакета до 50 сообщений → маршрутизация к обработчику → обновление смещения. `Lease`-блокировки хранятся в таблице `outbox_partition_locks` и предотвращают обработку одной партии несколькими экземплярами сервиса.

Очистка старых сообщений выполняется автоматически: ежедневно в 3:00 компонент `OutboxCleanupTask` удаляет сообщения старше 7 дней и истёкшие блокировки.

## 10.5 Мониторинг

Для мониторинга работы системы используется библиотека `Micrometer` с экспортом метрик в формат `Prometheus`. Метрики доступны по пути `/actuator/prometheus` на каждом сервисе.

Бизнес-метрики `core-transaction`:

- `outbox.producer.published` — счётчик опубликованных событий (теги: тип агрегата, тип события);
- `outbox.consumer.processed` — счётчик обработанных сообщений (теги: тип агрегата, тип события, статус);
- `outbox.consumer.lag` — задержка обработки сообщений по партициям;
- `outbox.consumer.lock.acquired` — результат захвата блокировки (успех/неудача);
- `outbox.consumer.errors` — количество ошибок обработки по партициям.

Бизнес-метрики `core-ledger`:



- `ledger.events.processed` — счётчик обработанных событий (теги: тип события, статус);
- `ledger.events.duplicates` — счётчик пропущенных дубликатов.

Помимо бизнес-метрик, Spring Boot Actuator автоматически собирает метрики HTTP-запросов, пула соединений HikariCP и JVM. Для визуального мониторинга Kafka-кластера используется Kafka UI, доступный по адресу <http://localhost:8080>.

## Вывод

По итогам курсовой работы у меня получилась полноценная событийно-ориентированная банковская платформа, состоящая из двух микросервисов на стеке Kotlin, Spring Boot, PostgreSQL и Apache Kafka. Я систематизировал материалы в единый документ: от описания предметной области и технического задания до структуры базы данных, архитектуры, набора UML-диаграмм, алгоритмов обработки транзакций, тестирования и справочного раздела с параметрами запуска.

Были сформулированы требования к системе и спроектирована модель данных с ER-схемой и SQL-миграциями, реализовано разделение ответственности по паттерну CQRS: сервис core-transaction отвечает за запись (проведение 8 типов банковских операций), а сервис core-ledger — за чтение (история операций, фильтрация, генерация отчётов в форматах PDF, DOCX и HTML). Для надёжной доставки событий между сервисами реализован паттерн Transactional Outbox с LIST-партиционированной таблицей, lease-блокировками и пакетной обработкой, что гарантирует атомарность записи бизнес-данных и событий, а также порядок обработки операций в рамках одного счёта.

Особое внимание уделено обеспечению корректности финансовых операций: реализована идемпотентность через уникальные ключи на обоих сервисах, пессимистичная блокировка счетов в алфавитном порядке для предотвращения deadlock-ов, ограничение неотрицательного баланса на уровне базы данных и ручное подтверждение Kafka-сообщений только после успешного сохранения в базу.

Написано 33 тестовых файла (21 в core-transaction, 12 в core-ledger), включающих модульные тесты с MockK и интеграционные тесты с Testcontainers (PostgreSQL и Kafka). Интеграционные тесты проверяют полные сценарии: от HTTP-запроса до записи в базу данных, включая асинхронное потребление Kafka-сообщений. Все тесты успешно пройдены.

Основные трудности — настройка взаимодействия между двумя сервисами через Kafka (гарантия порядка сообщений, обработка дубликатов, корректное смещение offset-ов), реализация партиционированной Outbox-таблицы с распределёнными блокировками, а также обеспечение консистентности данных при параллельных запросах к одним и тем же счетам.

В дальнейшем можно расширить платформу: добавить аутентификацию и авторизацию (Spring Security + JWT), реализовать Saga-оркестрацию для длительных транзакций с компенсациями, подключить визуализацию метрик через Grafana, добавить кэширование часто запрашиваемых данных (Redis), реализовать нотификации пользователей о статусе операций, а также подготовить Kubernetes-манифесты для промышленного развёртывания.

## **Список используемой литературы**

1. IEEE Recommended Practice for Software Requirements Specifications: IEEE Std 830-1998. - New York: Institute of Electrical and Electronics Engineers, 1998.
2. Ларман К. Применение UML и шаблонов проектирования. 2-е изд. - М.: Вильямс, 2006. - 720 с.
3. Spring Boot Reference Documentation [Электронный ресурс]. - Режим доступа: <https://docs.spring.io/spring-boot/documentation.html> (дата обращения: 29.03.2026).
4. Apache Kafka Documentation [Электронный ресурс]. - Режим доступа: <https://kafka.apache.org/documentation/> (дата обращения: 29.03.2026).
5. PostgreSQL Documentation [Электронный ресурс]. - Режим доступа: <https://www.postgresql.org/docs/> (дата обращения: 29.03.2026).
6. JUnit 5 User Guide [Электронный ресурс]. - Режим доступа: <https://junit.org/junit5/docs/current/user-guide/> (дата обращения: 29.03.2026).

## Приложение 1

### Листинг программы

```
private val log = KotlinLogging.logger {}

@Service
class OutboxProducerImpl(
    private val outboxMessageRepository: OutboxMessageRepository,
    private val objectMapper: ObjectMapper,
    private val outboxProperties: OutboxProperties,
    private val meterRegistry: MeterRegistry
) : OutboxProducer {

    override fun publish(
        aggregateType: OutboxAggregateType,
        aggregateId: String,
        eventType: String,
        partitionKey: String,
        payload: Any
    ) {
        if (!outboxProperties.producerEnabled) {
            log.debug { "Outbox producer is disabled, skipping:
type=$aggregateType, id=$aggregateId, event=$eventType" }
            return
        }

        val sample = Timer.start(meterRegistry)

        val partitionNum = abs(partitionKey.hashCode()) %
outboxProperties.partitionCount

        val jsonPayload = objectMapper.writeValueAsString(payload)
```

```

        val message = OutboxMessageEntity(
            aggregateType = aggregateType.name,
            aggregateId = aggregateId,
            eventType = eventType,
            partitionKey = partitionKey,
            partitionNum = partitionNum,
            payload = jsonPayload
        )
        outboxMessageRepository.save(message)

        sample.stop(
            Timer.builder("outbox.producer.publish.duration")
                .tag("aggregateType", aggregateType.name)
                .description("Time to serialize and persist outbox
message")
                .register(meterRegistry)
        )

        publishedCounter(aggregateType.name, eventType).increment()

        log.debug { "Outbox message published: type=$aggregateType,
id=$aggregateId, event=$eventType, partition=$partitionNum" }
    }

    private fun publishedCounter(aggregateType: String, eventType:
String) =

io.micrometer.core.instrument.Counter.builder("outbox.producer.publish
ed")

        .tag("aggregateType", aggregateType)
        .tag("eventType", eventType)
        .description("Total number of outbox messages published")
        .register(meterRegistry)

```

```
}
```

```
private val log = KotlinLogging.logger {}
```

```
@Service
```

```
class OutboxConsumerImpl(
```

```
    private val outboxMessageRepository: OutboxMessageRepository,
```

```
    private val offsetRepository: OutboxConsumerOffsetRepository,
```

```
    private val lockRepository: OutboxPartitionLockRepository,
```

```
    private val outboxProperties: OutboxProperties,
```

```
    private val transactionTemplate: TransactionTemplate,
```

```
    private val outboxHandlerRegistry: Map<String,  
    OutboxEventHandler>,
```

```
    private val meterRegistry: MeterRegistry
```

```
) : OutboxConsumer {
```

```
    private val instanceId = "${InetAddress.getLocalHost().hostname}-  
    ${UUID.randomUUID()}"
```

```
    private lateinit var executor: ScheduledExecutorService
```

```
    private val running = AtomicBoolean(false)
```

```
    private val offsetGauges = ConcurrentHashMap<Int, AtomicLong>()
```

```
    override fun isAutoStartup(): Boolean =  
    outboxProperties.consumerEnabled
```

```
    override fun start() {
```

```
        if (!outboxProperties.consumerEnabled) {
```

```
            log.info { "Outbox consumer is disabled" }
```

```
            return
```

```
        }
```

```

        log.info { "Starting outbox consumer: instanceId=$instanceId,
partitions=${outboxProperties.partitionCount},
handlers=${outboxHandlerRegistry.keys}" }

        running.set(true)

        executor =
Executors.newScheduledThreadPool(outboxProperties.partitionCount)

    for (partition in 0 until outboxProperties.partitionCount) {
        val offsetValue = AtomicLong(0)
        offsetGauges[partition] = offsetValue
        meterRegistry.gauge(
            "outbox.consumer.offset",
            listOf(Tag.of("partition", partition.toString())) ,
            offsetValue
        ) { it.toDouble() }

        executor.scheduleWithFixedDelay(
            { pollPartition(partition) },
            0,
            outboxProperties.pollIntervalMs,
            TimeUnit.MILLISECONDS
        )
    }
}

override fun stop() {
    if (!running.get()) return

    log.info { "Stopping outbox consumer: instanceId=$instanceId"
}

    running.set(false)
    executor.shutdown()

```



```

    try {
        executor.awaitTermination(10, TimeUnit.SECONDS)
    } catch (_: InterruptedException) {
        executor.shutdownNow()
    }

    for (partition in 0 until outboxProperties.partitionCount) {
        try {
            transactionTemplate.executeWithoutResult {

lockRepository.releaseLock(outboxProperties.consumerGroup, partition,
instanceId)

                }
            } catch (e: Exception) {
                log.warn(e) { "Failed to release lock for
partition=$partition" }
            }
        }
    }

    override fun isRunning(): Boolean = running.get()

    override fun getPhase(): Int = Int.MAX_VALUE - 1

    private fun pollPartition(partition: Int) {
        if (!running.get()) return

        val pollSample = Timer.start(meterRegistry)

        try {
            val pollResult = transactionTemplate.execute {

```

```

        val expiresAt =
OffsetDateTime.now().plusSeconds(outboxProperties.lockTtlSeconds)
        val acquired = lockRepository.tryAcquireLock(
            outboxProperties.consumerGroup, partition,
instanceId, expiresAt
        )

        lockCounter(partition, if (acquired > 0) "success"
else "failure").increment()

        if (acquired == 0) return@execute null

        val currentOffset = offsetRepository

.findByConsumerGroupAndPartitionNum(outboxProperties.consumerGroup,
partition)

        ?.lastOffset ?: 0L

        val messages =
outboxMessageRepository.findByPartitionAfterOffset(
            partition, currentOffset, PageRequest.of(0,
outboxProperties.batchSize)
        )

batchSizeSummary(partition).record(messages.size.toDouble())

        if (messages.isEmpty()) return@execute null

        PollResult(currentOffset, messages)
    } ?: run {
        pollSample.stop(pollDurationTimer(partition))
        return
    }
}

```

```

var lastSuccessId = pollResult.currentOffset
var processedCount = 0

for (message in pollResult.messages) {
    val handler =
outboxHandlerRegistry[message.aggregateType]
    if (handler == null) {
        log.warn { "No handler for
aggregateType=${message.aggregateType}, skipping
messageId=${message.id}" }

        processedCounter(message.aggregateType,
message.eventType, "skipped").increment()
        lastSuccessId = message.id
        processedCount++
        continue
    }

    try {
        val metadata = OutboxEventMetadata(
            messageId = message.id,
            aggregateType = message.aggregateType,
            aggregateId = message.aggregateId,
            partitionKey = message.partitionKey,
            partitionNum = message.partitionNum,
            createdAt = message.createdAt
        )

        val processSample = Timer.start(meterRegistry)
        handler.handle(message.eventType, message.payload,
metadata)

        processSample.stop(processDurationTimer(message.aggregateType,
message.eventType))
    }
}

```

```

        val lag = Duration.between(message.createdAt,
OffsetDateTime.now())
        lagTimer(partition).record(lag)

        processedCounter(message.aggregateType,
message.eventType, "success").increment()
        lastSuccessId = message.id
        processedCount++
    } catch (e: Exception) {
        log.error(e) { "Handler failed on
messageId=${message.id}, partition=$partition. Offset stays at
$lastSuccessId" }

        processedCounter(message.aggregateType,
message.eventType, "error").increment()
        break
    }
}

if (lastSuccessId > pollResult.currentOffset) {

offsetRepository.upsertOffset(outboxProperties.consumerGroup,
partition, lastSuccessId)

        offsetGauges[partition]?.set(lastSuccessId)
        log.debug { "Processed $processedCount messages from
partition=$partition, newOffset=$lastSuccessId" }
    }
} catch (e: Exception) {
    log.error(e) { "Error polling partition=$partition" }
    errorsCounter(partition).increment()
}

pollSample.stop(pollDurationTimer(partition))
}

```

```

    private fun processedCounter(aggregateType: String, eventType:
String, status: String) =
        Counter.builder("outbox.consumer.processed")
            .tag("aggregateType", aggregateType)
            .tag("eventType", eventType)
            .tag("status", status)
            .description("Total outbox messages processed by status")
            .register(meterRegistry)

    private fun processDurationTimer(aggregateType: String, eventType:
String) =
        Timer.builder("outbox.consumer.process.duration")
            .tag("aggregateType", aggregateType)
            .tag("eventType", eventType)
            .description("Handler processing time per message")
            .register(meterRegistry)

    private fun lagTimer(partition: Int) =
        Timer.builder("outbox.consumer.lag")
            .tag("partition", partition.toString())
            .description("Lag between message creation and
processing")
            .register(meterRegistry)

    private fun pollDurationTimer(partition: Int) =
        Timer.builder("outbox.consumer.poll.duration")
            .tag("partition", partition.toString())
            .description("Full poll cycle duration")
            .register(meterRegistry)

    private fun batchSizeSummary(partition: Int) =

```

```

        DistributionSummary.builder("outbox.consumer.poll.batch.size")
            .tag("partition", partition.toString())
            .description("Number of messages fetched per poll")
            .register(meterRegistry)

private fun lockCounter(partition: Int, result: String) =
    Counter.builder("outbox.consumer.lock.acquired")
        .tag("partition", partition.toString())
        .tag("result", result)
        .description("Lock acquisition outcomes")
        .register(meterRegistry)

private fun errorsCounter(partition: Int) =
    Counter.builder("outbox.consumer.errors")
        .tag("partition", partition.toString())
        .description("Unhandled errors in poll cycle")
        .register(meterRegistry)
}

```

## Приложение 2

### Проверка на антиплагиат

|                |        |               |       |                 |    |                 |    |             |  |                  |  |
|----------------|--------|---------------|-------|-----------------|----|-----------------|----|-------------|--|------------------|--|
| Оригинальность | 94,06% | Совпадения    | 6,94% | Цитирования     | 0% | Самоцитирования | 0% |             |  |                  |  |
| ПОЛНЫЙ ОТЧЕТ   |        | КРАТКИЙ ОТЧЕТ |       | ИСТОРИЯ ОТЧЕТОВ |    | РАСПЕЧАТАТЬ     |    | ↓ ВЫГРУЗИТЬ |  | 🔗 СОЗДАТЬ ССЫЛКУ |  |

📄 Свойства документа

Авторы документа ?

Не указано

Не указано

📁 Структура документа

Имя исходного файла

Милькин\_22-КБ-ПР1\_Курсовая\_ППК.pdf

Название документа

Милькин\_22-КБ-ПР1\_Курсовая\_ППК